

Database Design & Management

Structure Query Language

Lecturer: TẠ QUANG NGỌC

Email: tq.ngoc@vju.ac.vn

Phone: 0395648904

Class QR Code on MS Team



Đánh giá môn học

➤ Giữa kỳ (0.3):

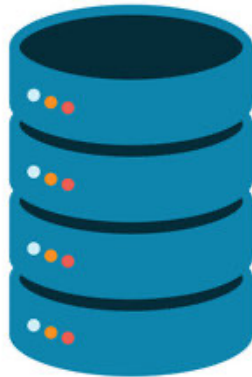
- 1-) Làm bài kiểm tra giữa kỳ (60 phút).
- 2-) Vắng mặt không quá 10% thời lượng học phần.
- 3-) Chuẩn bị tốt các phần được giao tự học và hoàn thành đúng hạn các bài tập được giao.
- 4-) Điều kiện cần để tham gia cuối kỳ.

➤ Cuối kỳ (0.7):

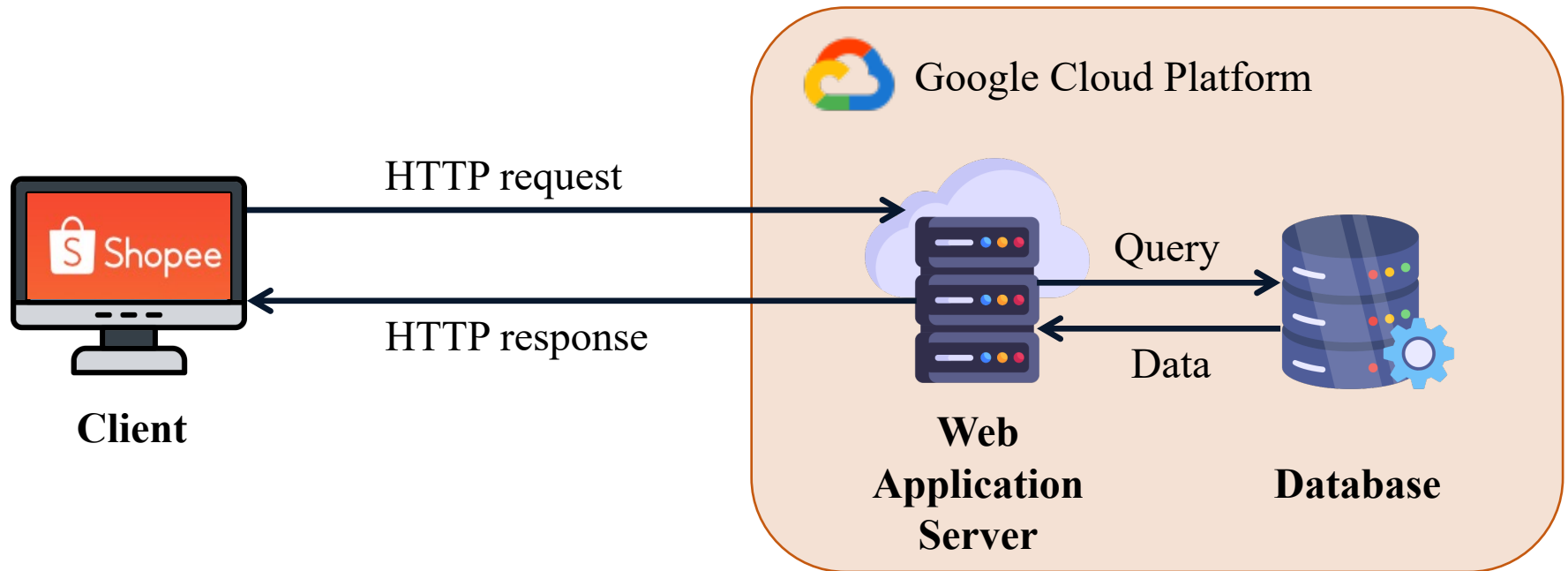
- 1-) Đã làm bài thi giữa kỳ.
- 2-) Thi trắc nghiệm + Tự luận.

Why Study This Course

Whether you know it or not, you
are using a database every day



Client-Server Model



Course Objective

Chapter 1: Overview of Database and MySQL

- Introduction to Database
- Database Management System (DBMS)
- MySQL, Workbench

Chapter 2: Relational Database Design

- Basics of Relational Databases
- Keys: Primary, Foreign, and Composite
- Database Normalization
- Entity-Relationship (ER) Modeling (Entities, Attributes, Relationships, ER Diagrams)

Chapter 3: Introduction to SQL

- What is SQL?
- SQL Syntax and Rules
- SQL Commands & Query Basics

Chapter 4: Working with MYSQL

- Workbench Interface

Week 1

Overview of Database and MySQL

What is Database?



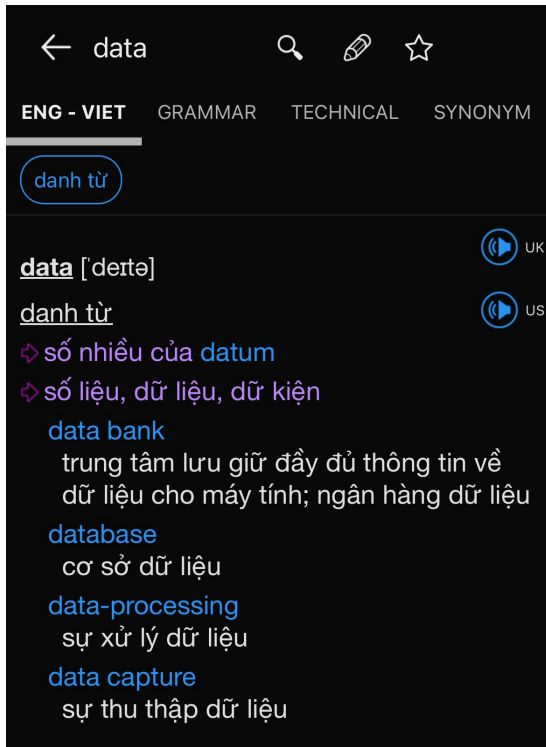
Data

+

Base

Definition: Database is an organized collection of structured information, or data, typically that is stored and accessed electronically in a computer system. It allows user to store, retrieve, and manage information. A database is usually controlled by a database management system (DBMS).

What is Data



Datum: Single piece of information

Data is a collection of facts, numbers, words, observations or other useful information.

- Text Document
- Images
- Audio
- Videos
- Software Program

Data is stored in the computer in a bunch of 0's and 1's (binary format)

- Created
 - Processed
 - Stored
- } Digitally
- 

This allow data to be transformed from one computer to another using the networking connection or various media devices

History of Database

1960s

Computerized Database

Charles Bachman designed the first database known as the Integrated Data Store, followed by the Information Management System developed by IBM.

1970s

Relational Model

E. F. Codd, a researcher at IBM, published a groundbreaking paper titled “A Relational Model of Data for Large Shared Data Banks”. Prototypes of relational database systems were developed, such as **Ingres** at the University of California and **System R** released by IBM.

1980s

SQL Standard

SQL became the industry standard query language, further solidifying the dominance of the relational model.

1990s

Database Software

The rise of the internet and the World Wide Web fueled a huge demand for client-server database systems, and the exponential growth of the database industry. MySQL was created in 1995, offering an alternative to the database systems of large companies like Oracle and Microsoft. Object-oriented database systems also grew more popular in this decade.

History of Database

2000s

NoSQL Database

The term NoSQL was coined in 1998, referring to databases that don't adhere to the traditional relational model. NoSQL databases are useful for unstructured data, allowing for faster processing of larger, more varied datasets.

Present

Modern Trends

- Cloud database: Cloud platforms offer various database services (e.g., Amazon RDS, Azure SQL database, Google Cloud SQL)
- NewSQL database: Combine the scalability of NoSQL with the properties of relational databases.
- Data warehousing and big data: specialized databases and technologies for storing and analyzing massive datasets.

Types of Database

➤ **Relational databases**

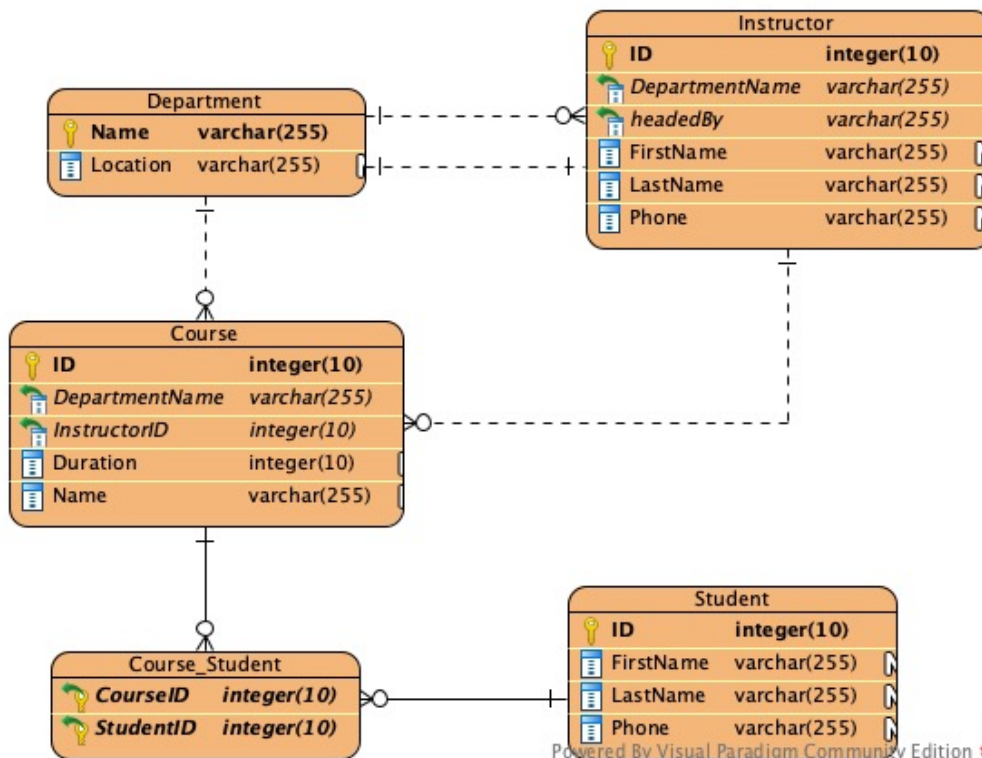
Relational databases are specifically designed for structured data. Items in a relational database are organized as a set of tables with columns and rows. Relational database technology provides the most efficient and flexible way to access structured information.

➤ **NoSQL databases**

NoSQL, or nonrelational database, allows unstructured and semistructured data to be stored and manipulated (in contrast to a relational database, which defines how all data inserted into the database must be in relation). NoSQL databases grew popular as web applications became more common and more complex.

Relational Databases

Relational database is based on the relational model and uses a collection of tables to represent both data and the relationship among those data. Entity Relationship Diagram is often be used to provide a visual representation of the data structure, making it easier to understand and communicate the database design.



NoSQL Databases

NoSQL databases store data differently than relational tables. NoSQL databases come in a variety of types based on their data model. The main types are document, key-value, wide-column, and graph. They provide flexible schemas and scale easily with large amounts of big data and high user loads.

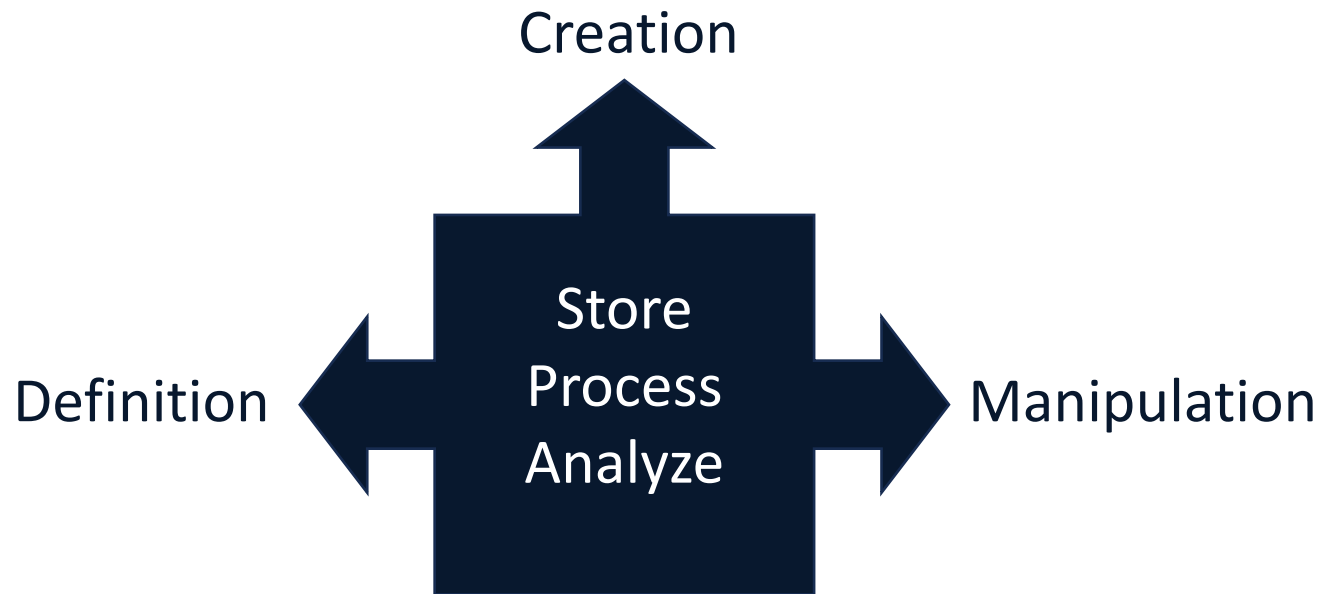
```
1  {  
2    "_id": "12345",  
3    "name": "foo bar",  
4    "email": "foo@bar.com",  
5    "address": {  
6      "street": "123 foo street",  
7      "city": "some city",  
8      "state": "some state",  
9      "zip": "123456"  
10   },  
11   "hobbies": ["music", "guitar", "reading"]  
12 }
```



```
1  Key: user:12345  
2  Value: {"name": "foo bar", "email": "foo@bar.com", "designation": "software  
       developer"}
```

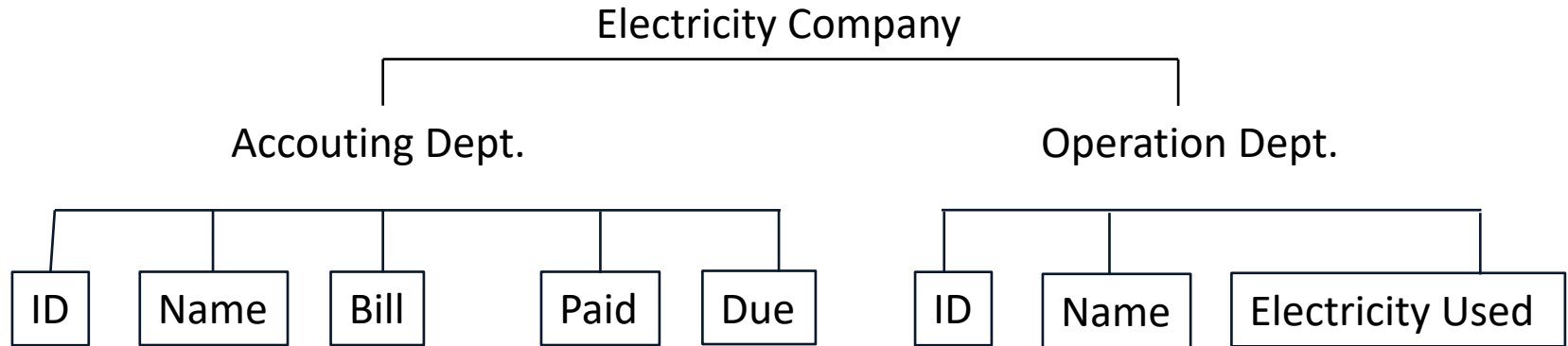
Database Management System (DBMS)

DBMS is software that allows creation, definition and manipulation of database, which allows users to store, process, analyze data. DBMS also manages the security and access controls for that database. DBMS provides an environment to store and retrieve data in convenient and efficient manner.

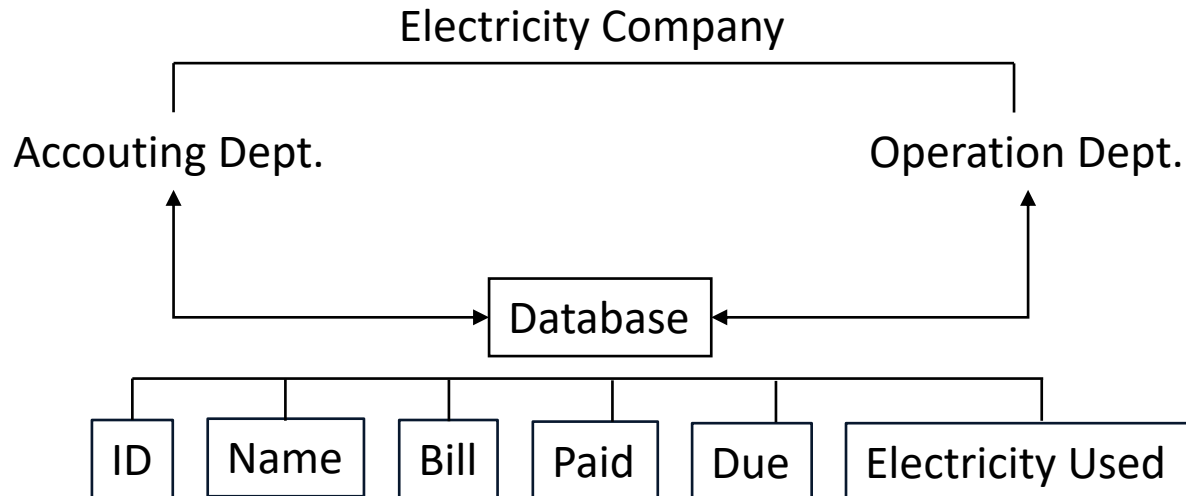


Database Management System (DBMS)

➤ File System



➤ DBMS



- All data store in one place
- Everyone working in the same system
- No duplication, fewer error
- Complete business clarity

DBMS Applications

DBMS are used almost in every software that we use today. For example, Emails, WhatsApp, Social Media. Almost all programming languages provide libraries from different popular database management systems like MySQL, Oracle, Microsoft SQL Server, MongoDB, etc.

- **Banking:** Manages accounts, transactions, and financial records.
- **Airlines:** Handles bookings, schedules, and availability.
- **E-Commerce:** Supports catalogs, orders, and secure transactions.
- **Healthcare:** Stores patient records and billing.
- **Education:** Manages student data and course enrollments.
- **Telecom:** Track call records and billing.
- **Government:** Maintains census and taxation data.
- **Social Media:** Stores user profiles and posts efficiently.

DBMS Features

- **Data modeling:** A DBMS provides tools for creating and modifying data models, which define the structure and relationships of the data in a database.
- **Data storage and retrieval:** A DBMS is responsible for storing and retrieving data from the database, and can provide various methods for searching and querying the data.
- **Concurrency control:** A DBMS provides mechanisms for controlling concurrent access to the database, to ensure that multiple users can access the data without conflicting with each other.
- **Data integrity and security:** A DBMS provides tools for enforcing data integrity and security constraints, such as constraints on the values of data and access controls that restrict who can access the data.
- **Backup and recovery:** A DBMS provides mechanisms for backing up and recovering the data in the event of a system failure.

Database Management System (DBMS)



Introduction to MySQL

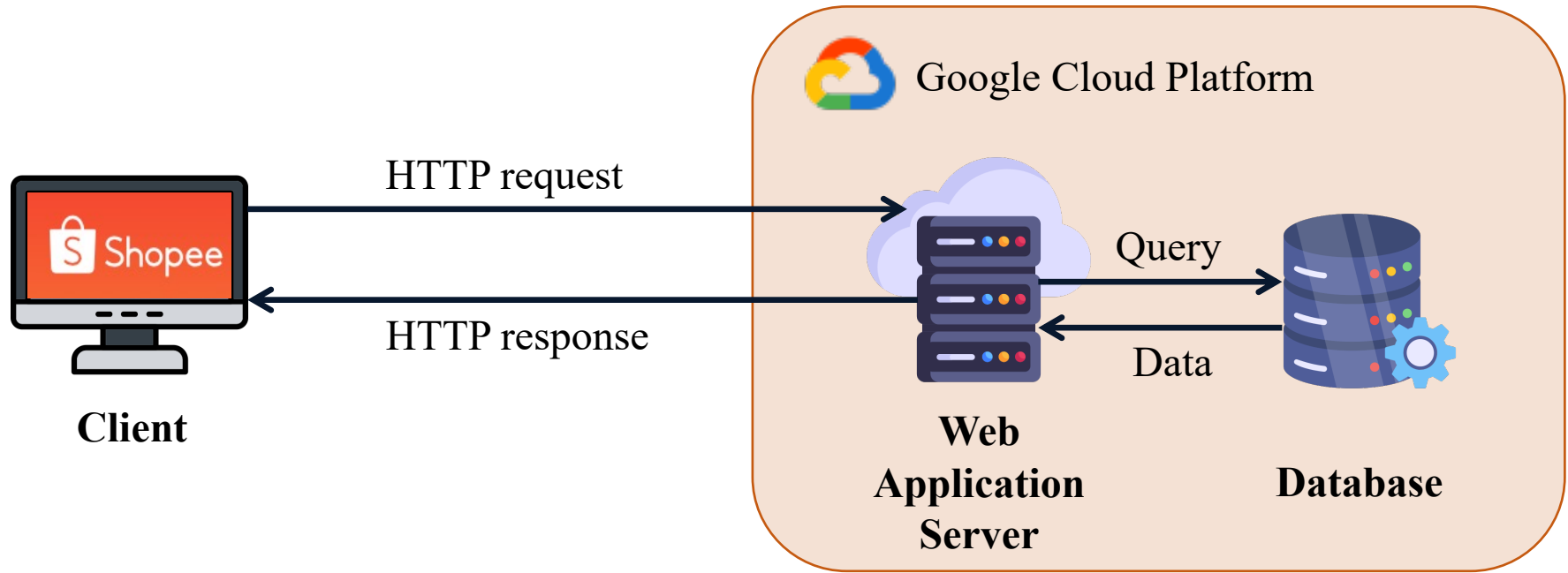
MySQL is a popular open-source Relational Database Management System that use structure query language (SQL) for database operations. MySQL is a specific database system accessible for free and supports various programming languages. MySQL is widely used for various applications, from small websites to large-scale enterprise systems.

➤ Why use MySQL?

- **Open Source:** MySQL is open-source software, which means it's free to use and has a large community of developers contributing to its improvement.
- **Relational:** MySQL follows the relational database model, allowing users to organize data into tables with rows and columns, facilitating efficient data storage and retrieval.
- **Reliability:** MySQL is optimized for performance, making it capable of handling high-volume transactions and large datasets efficiently.
- **Scalability:** MySQL can scale both vertically and horizontally to accommodate growing data and user loads. You can add more resources to a single server or distribute the workload across multiple servers using techniques like sharding or replication.
- **Compatibility:** MySQL is widely supported by many programming languages, frameworks, and tools. It offers connectors and APIs for popular languages like PHP, Python, Java,..., making it easy to integrate with your existing software stack.
- **Security:** MySQL provides robust security features to protect your data, including access controls, encryption, and auditing capabilities. With proper configuration, you can ensure that only authorized users have access to sensitive information.

How MySQL Works?

MySQL follows the working of Client-Server Architecture. This model is designed for the end-users called client to access the resources from a central computer known as a server using network services. Here, the clients make request through a graphical user interface (GUI), and the server will give the desired output as soon as the instructions are matched. The process of MySQL environment is the same as the client-server model.



How MySQL Works?

The core of the MySQL database is the MySQL Server. This server is available as a separate program and responsible for handling all the database instructions, statements, or commands. The working of MySQL database with MySQL Server are as follows:

- MySQL creates a database that allows you to build many tables to store and manipulate data and defining the relationship between each table.
- Clients make requests through the GUI screen or command prompt by using specific SQL expression on MySQL.
- Finally, the server application will respond with the request expressions and produce the desired result on the client-side.

A client can use any MySQL GUI. But, it is making sure that your GUI should be lighter and user-friendly to make your data management activities faster and easier. Some of the most widely used MySQL GUIs are MySQL Workbench, SequelPro, DBVisualizer, and the Navicat DB Admin Tool.

What is MySQL Workbench?

MySQL Workbench is a unified cross-platform, open-source relational database design tool that adds functionality and ease to MySQL and SQL development work. MySQL Workbench provides data modeling, SQL development, and various administration tools for configuration. It also offers a graphical interface to work with the databases in a structured way.

- Be able to create a Graphical Model using MySQL Workbench
- MySQL Workbench provides reverse engineering for live databases to models
- MySQL Workbench offers a forward engineering model to a script/live database

MySQL Workbench – Modeling & Designing Tool

- MySQL Workbench possesses tools that allow database administrators to virtually create physical database design models that can be easily transitioned into MySQL databases using forward engineering.
- MySQL Workbench adheres to all objects such as tables, views, stored procedures, triggers, etc.
- MySQL Workbench also creates models from a target database or even imported SQL files.

MySQL Workbench – SQL Development Tool

- MySQL Workbench comes with a visual SQL editor.
- The Visual SQL editor gives developers the access to build, edit, and run queries against MySQL server databases. It has utilities for viewing data and exporting it.
- MySQL Workbench has auto-complete and color highlighters that aid in the writing and debugging of SQL statements, easily.
- Multiple queries can be run at a time, and the result is automatically displayed.
- It also saves the queries in the history panel for previewing and running it later on.

MySQL Workbench – SQL Administration Tool

- Be able to view the account information of all users on the MySQL server.
- MySQL Workbench gives access to add and remove users.
- MySQL Workbench grants and revokes privileges.
- You can modify global and database permissions on the MySQL server.
- Be able to change passwords using MySQL.
- You can audit to see who did what and when on the server

How to Install MySQL Workbench?

- **Step 1:** Download & Install MySQL Community Server.
(<https://dev.mysql.com/downloads/>).
- **Step 2:** Download & Install MySQL Workbench.
(<https://dev.mysql.com/downloads/workbench/>).
- **Step 3:** Connecting Workbench & Community Server

Week 2

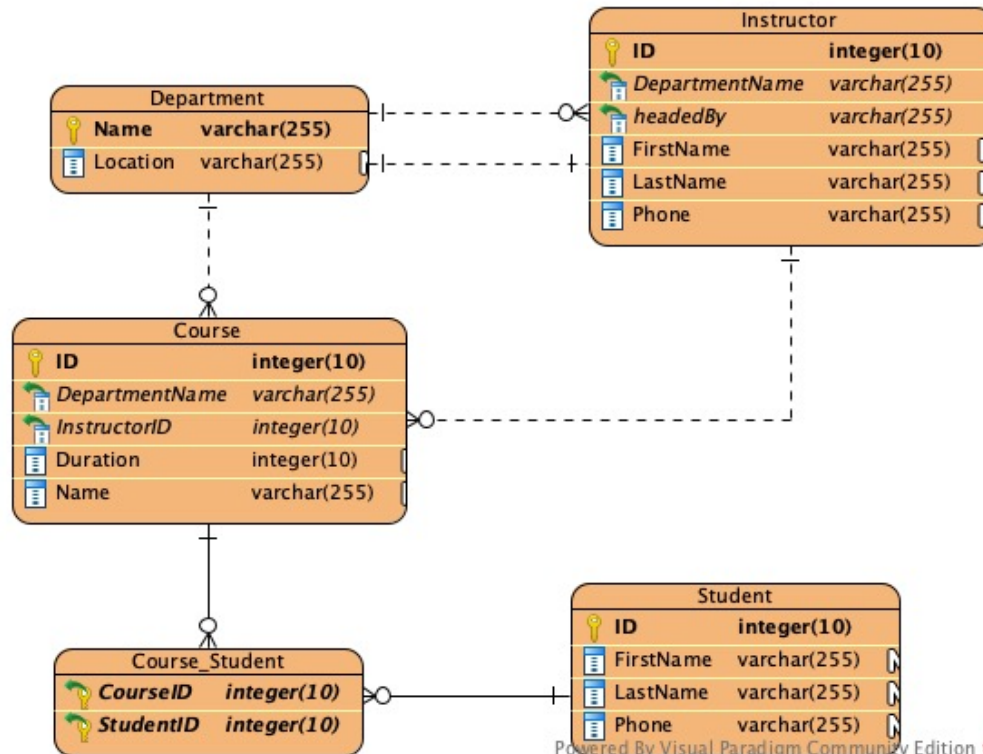
Relational Database Design

Relational Database

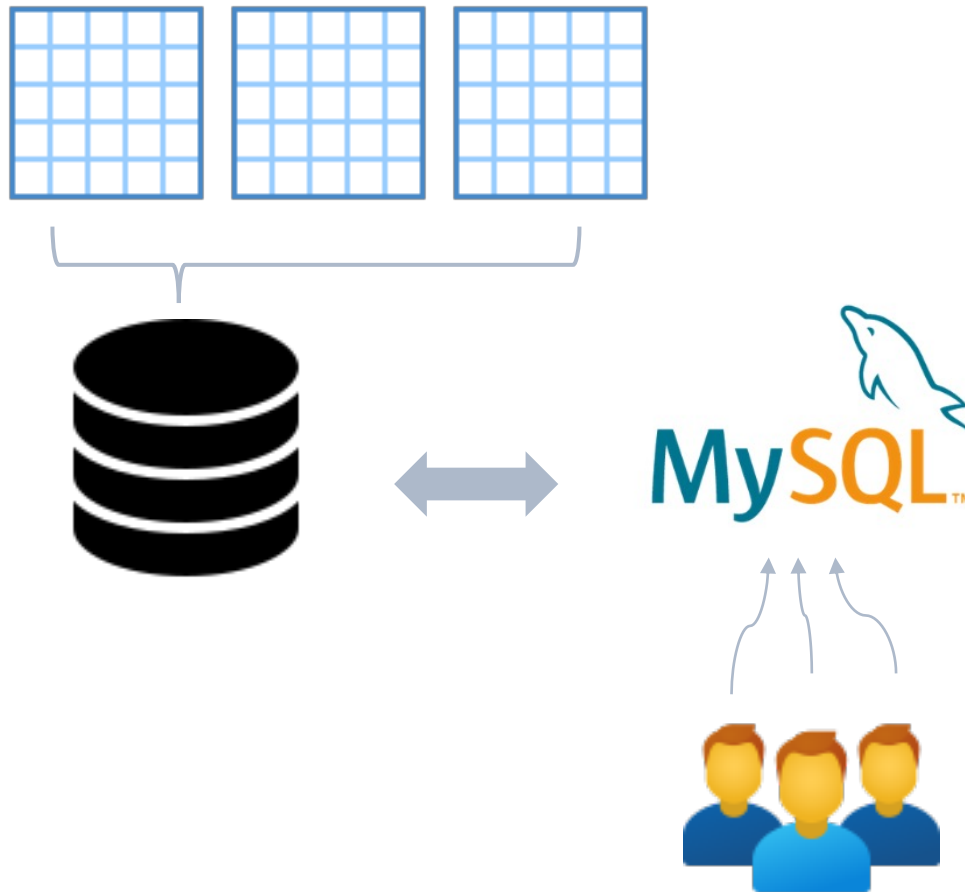
- Basics of Relational Databases
- Keys: Primary, Foreign
- Entity-Relationship (ER) Modeling (Entities, Attributes, Relationships, ER Diagrams)
- Database Normalization

What is a Relational Database?

A relational database is a collection of information that organizes data in predefined relationship where data is stored in one or more tables (or “relations”) of columns and rows, making it easy to see and understand how different data structures relate to each other. Relationships are a logical connection between different tables, established on the basis of interaction among these tables.



Relational Database Management System (RDBMS)



- Relational databases consist of data stored in multiple **tables**.
- **RDBMS** serves as an interface to access and manage relational database, using SQL.
- RDBMS administrators can grant access to **users** and set specific roles and permissions.

Database Terms

- **Entity:** an entity represents a real-world object or concept that we want to store information about in the database. (Ex: customer, product, student, employee,...)
- **Attribute:** a characteristic or property of an entity. (Ex: "student" entity: ID, name, phone)
- **Table:** a two-dimensional structure with rows and columns used to store data in a relational database. Each column represent an attribute of the entity. Each row represents a record of an entity.
- **Primary Key:** a unique identifier for each row in a table. It ensures that each row is distinct. A primary key cannot contain null values
- **Foreign Key:** a column in one table that references the primary key of another table. Foreign keys establish relationship between tables.
- **Relationship:** a relationship defines how two or more entities are associated with each other.

Database Terms

Entity

Student

Attribute

Attribute

Attribute

Attribute

Attribute

ID	Name	Phone	Email	Bank
0	Nguyen Thi A	123456	nta@gmail.com	Vietcombank
1	Le Van B	234567	lvb@gmail.com	Techcombank
2	Le Thi C	345678	lth1@gmail.com	BIDV
3	Le Thi C	456789	lth2@gmail.com	Vietcombank
4	Nguyen Van A	765432	nva@gmail.com	Vietinbank
5	Le Van D	876543	lvd@gmail.com	Saigonbank

Database Terms

(Many)
Foreign
Key

(Many)
Foreign
Key

inventory_id	film_id	address_id
1	1	1
2	1	1
3	1	1
4	1	1
5	1	2
6	1	2
7	1	2
8	1	2
9	2	2
10	2	2
11	2	2
12	3	2
13	3	2
14	3	2
15	3	2
16	4	1
17	4	1
18	4	1
19	4	1
20	4	2

Primary
Key

(One)

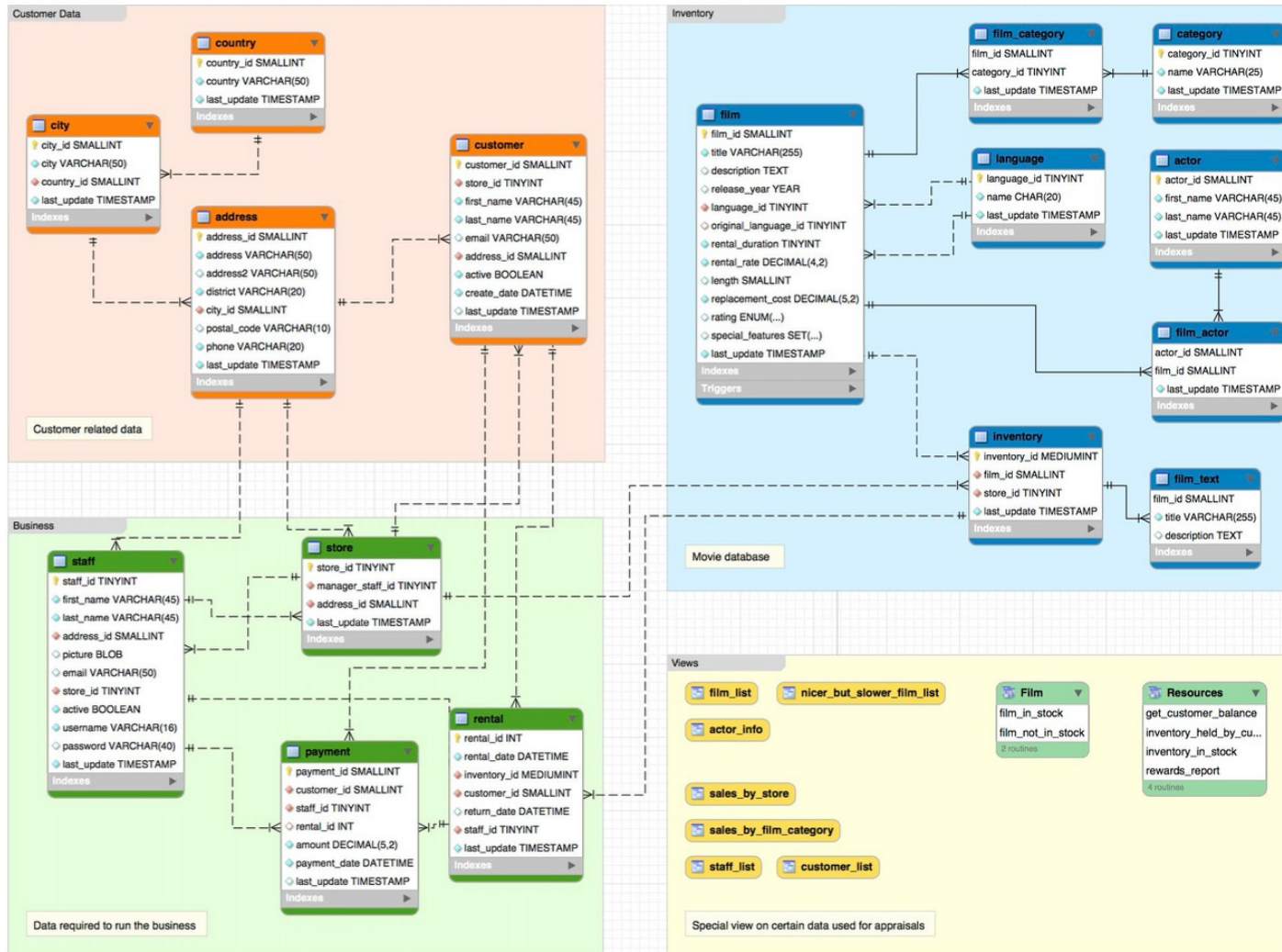
film_id	title	release
1	Nguoi Sat	2000
2	The Gioi Dong Vat	2010
3	Ke Huy Diet	2002
4	Quy Ngai Gasby	2005

Primary
Key

(One)

address_id	address	district
1	68 Xuan Thuy	Hanoi
2	92 Pham Van Hai	Ho Chi Minh

Database Terms

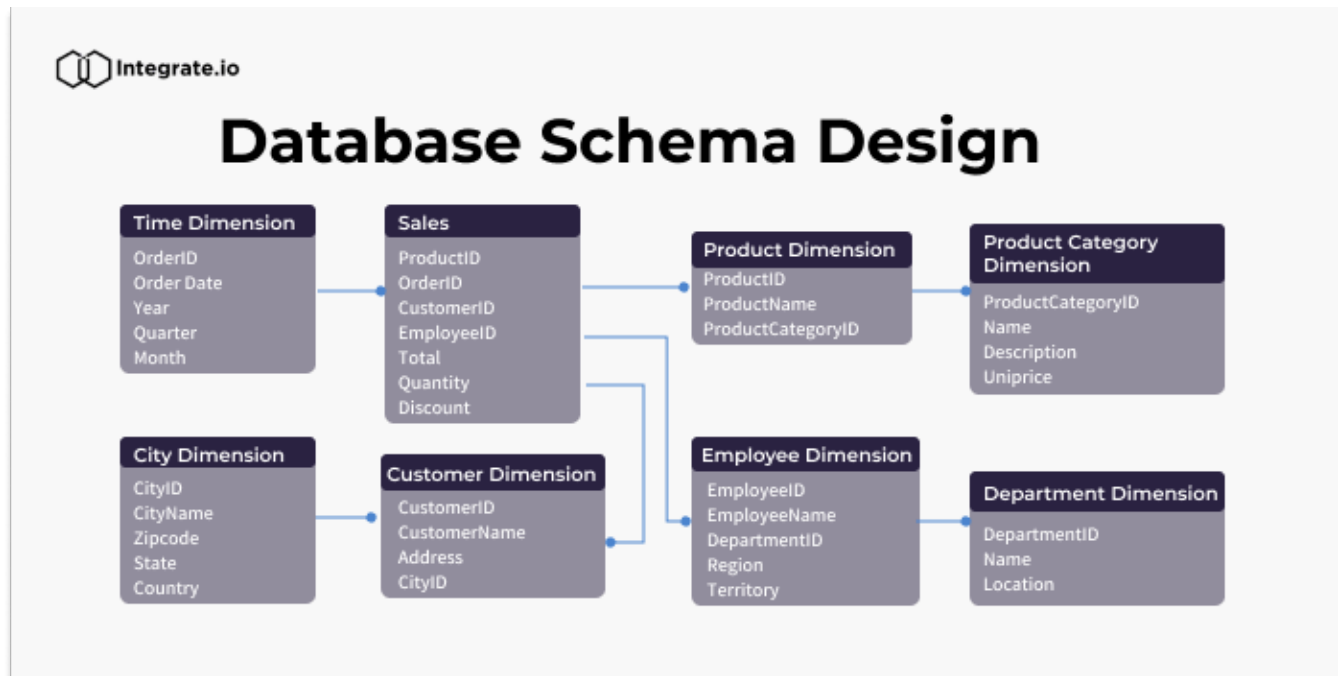


Database Terms

- **Join:** An operation that combines rows from two or more tables based on a related column.
- **Index:** a data structure that improves the performance of data retrieval operations by creating a sorted representation of the data in a table.
- **View:** A virtual table that is dynamically generated from one or more underlying tables.
- **Stored Procedure:** a pre-compiled collection of SQL statements that can be executed as a single unit.
- **Trigger:** a special type of stored procedure that is automatically executed when a specific event occurs in a table, such as an INSERT, UPDATE, or DELETE statement.

What is Database Design?

Database design is the process of creating an efficient and organized structure for storing and managing data in a database. It involves defining tables, columns, relationships, and constraints to ensure data integrity, minimize redundancy, and optimize performance. Proper database design is the foundation for building robust and scalable applications.



src: <https://dev.to/louaiboumediene/mastering-relational-database-design-a-comprehensive-guide-3jh8>

Data Integrity

Data integrity refers to the accuracy, consistency, and reliability of data stored in a database. It ensures that the data follows specific rules and constraints, preventing errors and inconsistencies. There are three types of data integrity:

1. **Entity Integrity:** This ensures that there are no duplicate rows in a table.
2. **Referential Integrity:** Rows cannot be deleted, which are used by other records.
3. **Domain Integrity:** Enforces valid entries for a given column by restricting the data type, format, and range of values that can be stored.

What is Database Normalization?

Database normalization is the process of structuring the tables and columns in a relational database. There are two reasons of this normalization process:

- Eliminating redundant data (Ex: storing the same data in more than one table).
- Ensuring data dependencies make sense.

This is to make sure that they reduce the amount of space a database consumes and ensures that data is logically stored. Normalization consists of a series of guidelines that help guide you in creating a good database structure.

Normalization guidelines are divided into normal forms, which are the format or the way a database structure is laid out. The aim of normal forms is to organize the database structure, so that it complies with the rules of

- First normal form (1NF)
- Second normal form (2NF)
- Third normal form (3NF)

What is Database Normalization?

	NAME OF THE STUDENT	UNIVERSITY	PROGRAM NAME	Specialisation	SEMESTER	Domain	GENERAL MARK	Domain Specialisation
0	Camila Wood	Stanford University, USA	B.Com	Honours	5th	Finance	50	50
1	Alexander Thompson	Stanford University, USA	B.Com	Financial Services	5th	Finance	50	50
2	Liam Taylor	Harvard University, USA	B.Com	Accounting Analytics	5th	BA	50	50
3	Evelyn Jenkins	Stanford University, USA	B.Com	Honours	5th	Finance	49	50
4	Michael Jackson	Harvard University, USA	MBA	International Business	3rd	IB	50	49
5	Chloe Moore	Stanford University, USA	B.Com	Honours	5th	Finance	50	48
6	Nicholas Clark	Harvard University, USA	MBA	International Business	3rd	IB	49	49
7	Olivia Richardson	Stanford University, USA	BBA	International Business	5th	IB	50	47
8	Aiden Rivera	Stanford University, USA	B.Com	Honours	5th	Finance	50	47
9	Harper Turner	Harvard University, USA	MBA	International Business	3rd	IB	49	48
10	Madison Lee	Stanford University, USA	BBA	Business Analytics	5th	BA	47	50
11	Carter Hall	Harvard University, USA	MBA	International Business	3rd	IB	50	47
12	Mia Thomas	Harvard University, USA	B.Com	Information Technology with Accounting Analytics	5th	BA	50	47
13	Lucas Evans	University of Oxford, UK	Intg. BBA+MBA	Financial Services	9th	Finance	49	47
14	Abigail Bailey	Harvard University, USA	MBA	International Business	3rd	IB	47	48
15	Joseph Diaz	Harvard University, USA	MBA	International Business	3rd	IB	49	46
16	William Walker	University of Cambridge, UK	MBA	Information Technology with Accounting Analytics	5th	BA	48	47
17	Andrew Anderson	University of Oxford, UK	Intg. BBA+MBA	Financial Services	9th	Finance	48	47
18	Sophia King	Harvard University, USA	B.Com	Information Technology with Accounting Analytics	5th	BA	48	46
19	Avery Gonzalez	Harvard University, USA	MBA	Innovation Entrepreneurship and Venture Development	3rd	E&I	49	43
20	Lily Davis	Harvard University, USA	MBA	Innovation Entrepreneurship and Venture Development	3rd	E&I	49	43
21	Scarlett Cooper	Stanford University, USA	BBA	International Business	5th	IB	49	43
22	Grace Reed	Stanford University, USA	BBA	International Business	5th	IB	49	43
23	Matthew Johnson	Harvard University, USA	MBA	International Business	3rd	IB	49	43
24	Jackson Baker	Stanford University, USA	BBA	International Business	5th	IB	45	47
25	Ella Martinez	Harvard University, USA	B.Com	Information Technology with Accounting Analytics	5th	BA	49	43
26	Henry Perez	Stanford University, USA	B.Com	Honours	5th	Finance	41	50
27	Ava White	Stanford University, USA	BBA	International Business	5th	IB	45	46
28	Gabriel Miller	Stanford University, USA	B.Com	Honours	5th	Finance	44	46
29	Mia Brown	Stanford University, USA	BBA	Financial Services	5th	Finance	41	49
30	Layla Garcia	Harvard University, USA	B.Com	Information Technology with Accounting Analytics	5th	BA	48	42
31	Emma Stewart	Stanford University, USA	BBA	Financial Services	5th	Finance	40	50
32	Daniel Adams	University of Oxford, UK	Intg. BBA+MBA	Financial Services	9th	Finance	47	43
33	Logan Smith	Stanford University, USA	BBA	International Business	5th	IB	45	45
34	James Harris	Stanford University, USA	B.Com	Accounting	5th	Finance	42	47
35	Benjamin Martin	Harvard University, USA	MBA	International Business	3rd	IB	48	41

In practice, normalization involves breaking out data from a **single merged table** into **multiple related tables**

What is Database Normalization?

Not normalized

inventory_id	title	release	store_address	store_district
1	Nguoi Sat	2000	68 Xuan Thuy	Hanoi
2	Nguoi Sat	2000	68 Xuan Thuy	Hanoi
3	Nguoi Sat	2000	68 Xuan Thuy	Hanoi
4	Nguoi Sat	2000	68 Xuan Thuy	Hanoi
5	Nguoi Sat	2000	92 Pham Van Hai	Ho Chi Minh
6	Nguoi Sat	2000	92 Pham Van Hai	Ho Chi Minh
7	Nguoi Sat	2000	92 Pham Van Hai	Ho Chi Minh
8	Nguoi Sat	2000	92 Pham Van Hai	Ho Chi Minh
9	The Gioi Dong Vat	2010	92 Pham Van Hai	Ho Chi Minh
10	The Gioi Dong Vat	2010	92 Pham Van Hai	Ho Chi Minh
11	The Gioi Dong Vat	2010	92 Pham Van Hai	Ho Chi Minh
12	Ke Huy Diet	2002	92 Pham Van Hai	Ho Chi Minh
13	Ke Huy Diet	2002	92 Pham Van Hai	Ho Chi Minh
14	Ke Huy Diet	2002	92 Pham Van Hai	Ho Chi Minh
15	Ke Huy Diet	2002	92 Pham Van Hai	Ho Chi Minh
16	Quy Ngai Gasby	2005	68 Xuan Thuy	Hanoi
17	Quy Ngai Gasby	2005	68 Xuan Thuy	Hanoi
18	Quy Ngai Gasby	2005	68 Xuan Thuy	Hanoi
19	Quy Ngai Gasby	2005	68 Xuan Thuy	Hanoi
20	Quy Ngai Gasby	2005	92 Pham Van Hai	Ho Chi Minh

What is Database Normalization?

Normalized

inventory_id	film_id	address_id
1	1	1
2	1	1
3	1	1
4	1	1
5	1	2
6	1	2
7	1	2
8	1	2
9	2	2
10	2	2
11	2	2
12	3	2
13	3	2
14	3	2
15	3	2
16	4	1
17	4	1
18	4	1
19	4	1
20	4	2

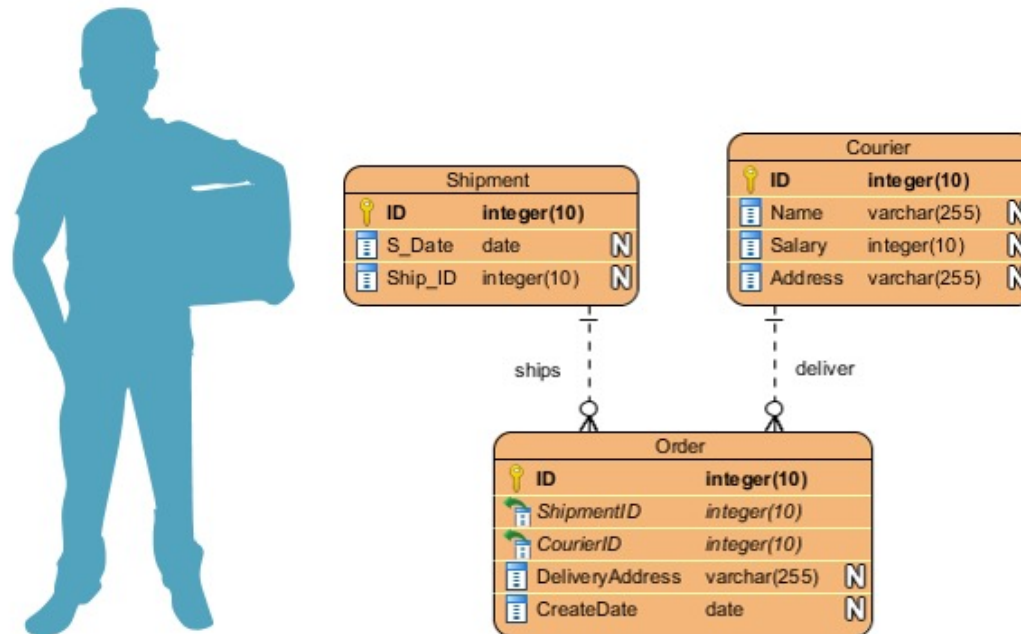
film_id	title	release
1	Nguoi Sat	2000
2	The Gioi Dong Vat	2010
3	Ke Huy Diet	2002
4	Quy Ngai Gasby	2005

address_id	address	district
1	68 Xuan Thuy	Hanoi
2	92 Pham Van Hai	Ho Chi Minh

What is an Entity-Relationship Diagram (ERD)?

An entity relationship diagram (ERD) is a visual representation of the entities, attributes, and relationships within a database. It helps in designing the database schema.

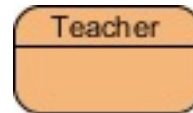
An ERD contains different symbols and connectors that visualize two important information: The major entities within the system scope, and the inter-relationships among these entities.



ERD Notations

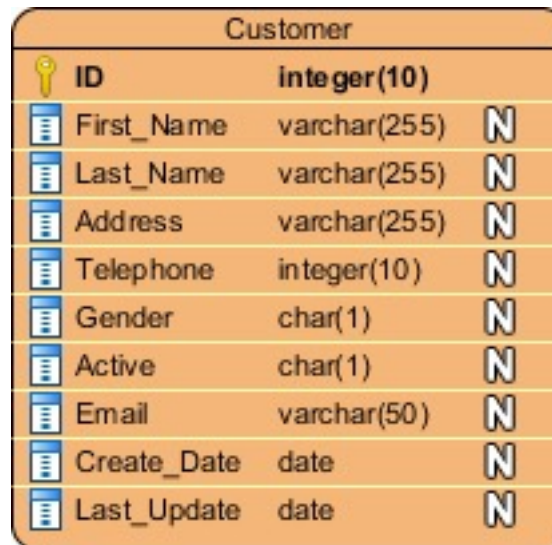
Entity is a definable thing or concept within a system, for example

- Person/Role: Teacher, Student
- Object: Invoice, Order
- Concept: Profile
- Event: Transaction



Entity Attributes is a property or characteristic of the entity that holds it. For example, a customer may have attributes as following:

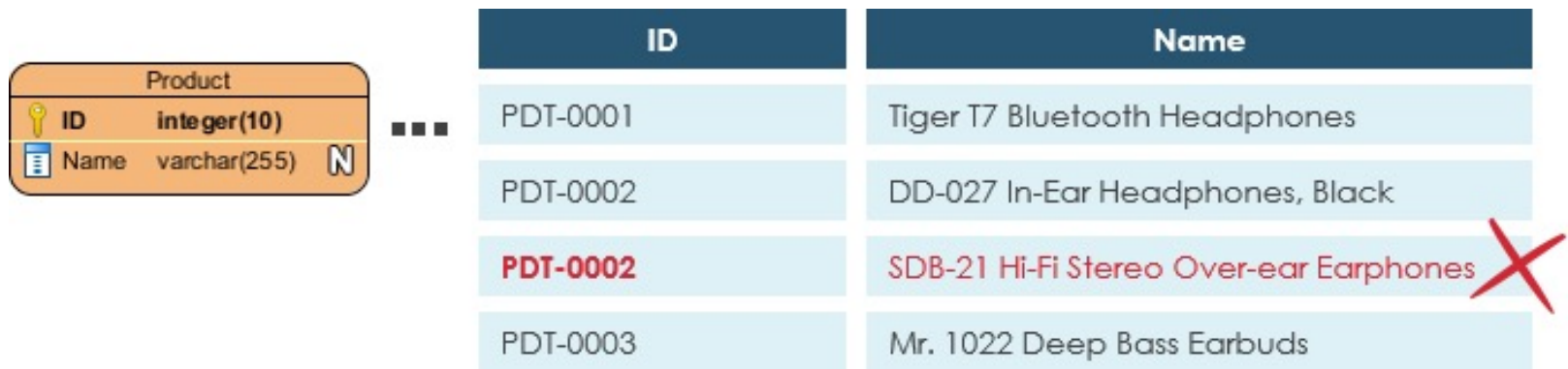
- ID
- First_Name
- Last_Name
- Address
- Telephone
- Gender
- Active
- Email
- Create_Date
- Last_Update



ERD Notations

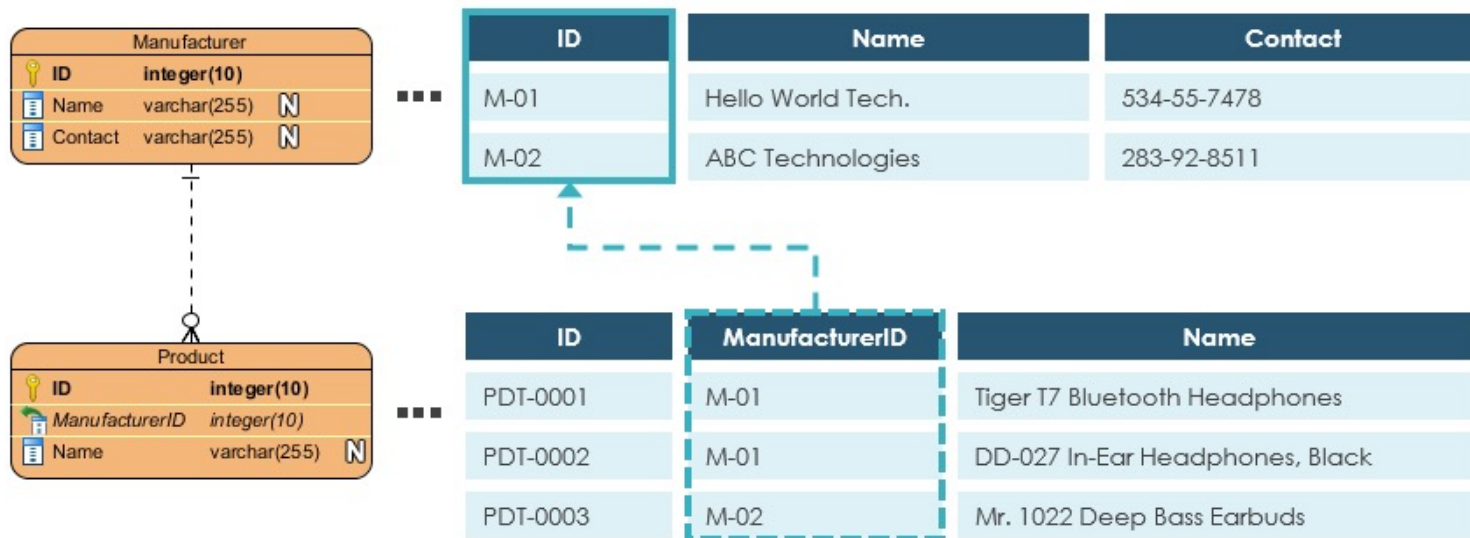
Primary Key (PK) is a special kind of entity attribute that uniquely defines a record in a database table. In other words, there must not be two (or more) records that share the same value for the primary key attribute.

Example: The ERD example below shows an entity “**Product**” with a primary key attribute “**ID**”, and a preview of table records in the database. The third record is invalid because of ID “PDT-0002” is already used by another record.



ERD Notations

Foreign Key (FK) is a reference to a primary key in a table. It is used to identify the relationships between entities. Note that foreign keys need not be unique. Multiple records can share the same values. The ER Diagram example below shows an entity with some columns, among which a foreign key is used in referencing another entity.



Relationship

Relationship defines how two or more entities are associated with each other. There are three main types of relationships:

- **One-to-One Relationship:** is mostly used to split an entity in two to provide information concisely and make it more understandable.
- **One-to-Many Relationship:** refers to the relationship between two entities X and Y in which an instance of X may be linked to many instances of Y, but an instance of Y is linked to only one instance of X.
- **Many-to-Many Relationship:** refers to the relationship between two entities X and Y, in which X may be linked to many instances of Y and vice versa. Note that a many-to-many relationship is split into a pair of one-to-many relationships in a physical ERD.

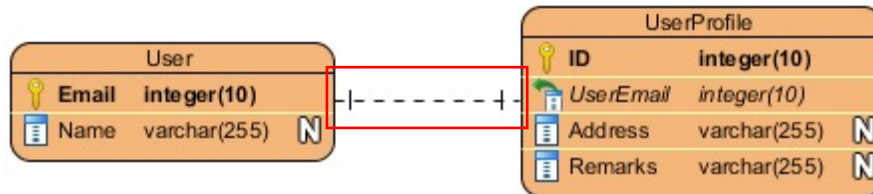
Cardinality

Cardinality defines the possible number of occurrences in one entity which is associated with the number of occurrences in another.

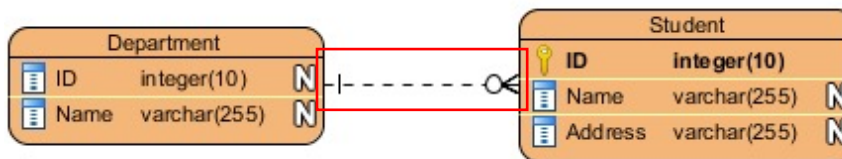
Example: ONE Team has MANY Players. When present in an ERD, the entity Team and Player are inter-connected with a one-to-many relationship.

Cardinality

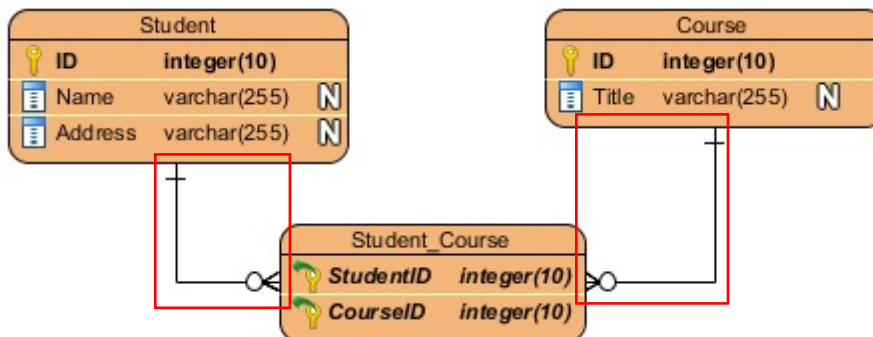
- One-to-One Relationship:



- One-to-Many Relationship:



- Many-to-Many Relationship:



Cardinality

—| One

—< Many

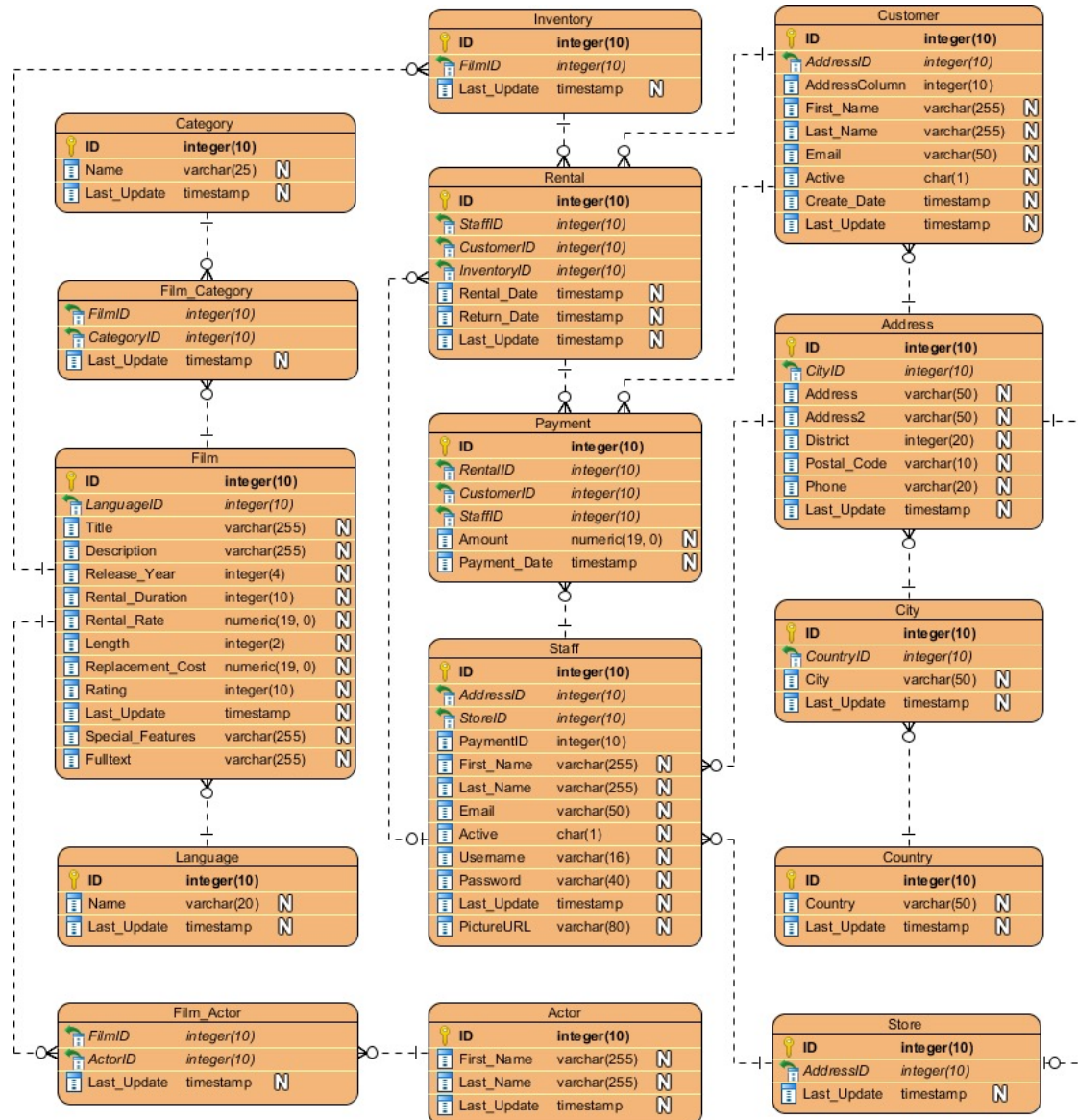
—|| One (and only one)

—○| Zero or one

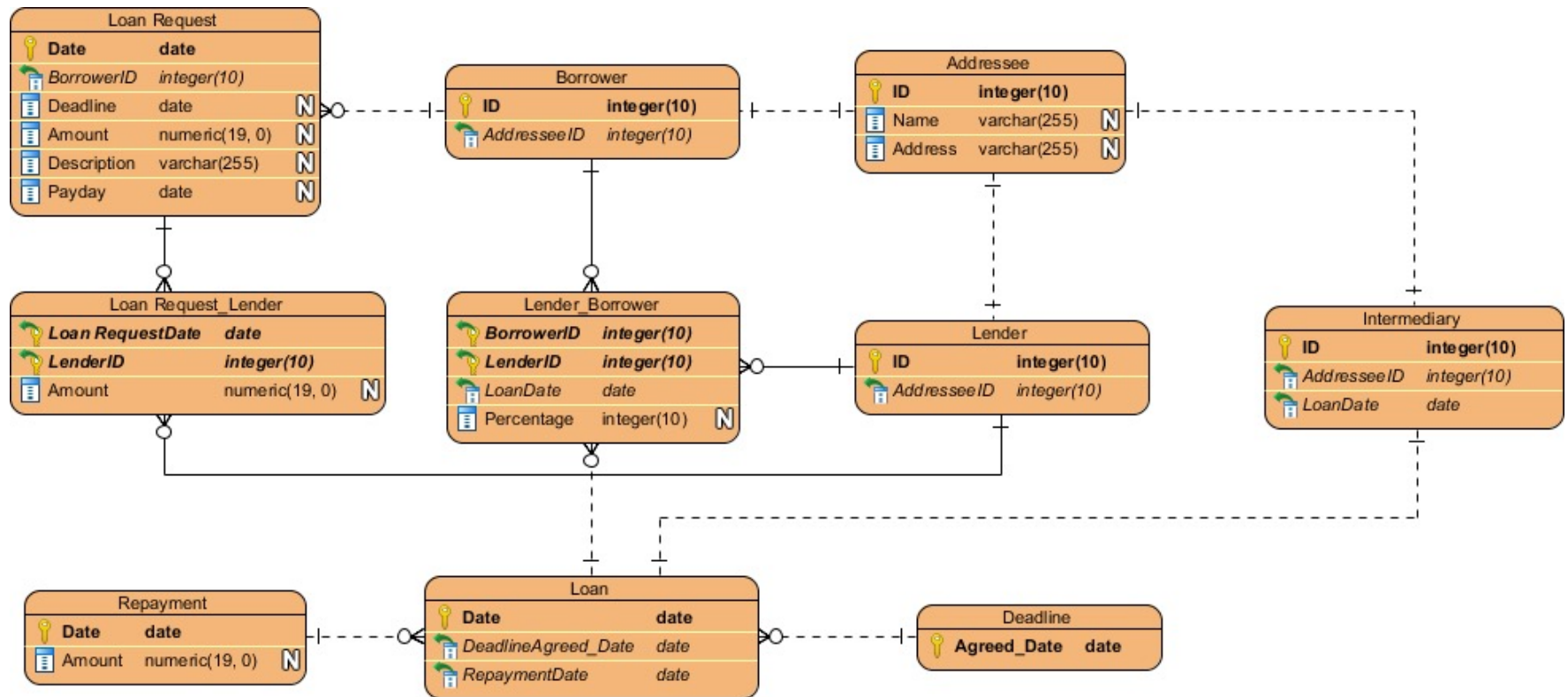
—|< One or many

—○< Zero or many

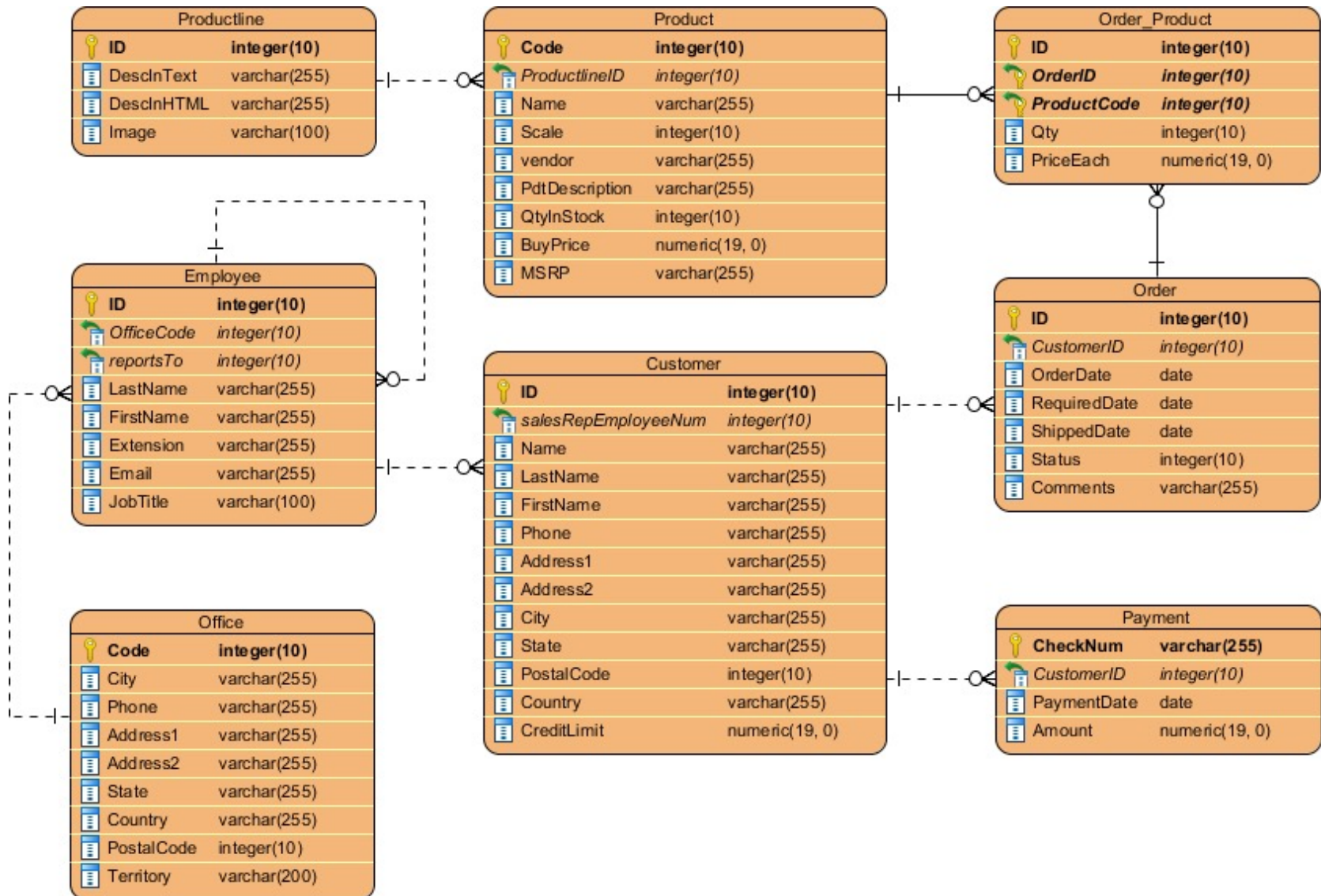
ERD – Movie Rental System



ERD – Loan System



ERD – Online Shop



ERD – Online Shopping System

Entities

- Product (attributes: product ID, name, price, category, stock quantity)
- Customer (attributes: customer ID, name, email, address)
- Order (attributes: order ID, order date, total amount)
- Payment (attributes: payment ID, payment date, payment method)

Relationship

- A customer can place multiple orders
- An order can contain multiple products
- Each order is associated with one payment

ERD – Hospital Management System

Entities

- Patient (attributes: patient ID, name, date of birth, gender)
- Doctor (attributes: doctor ID, name, specialization)
- Appointment (attributes: appointment ID, date, time)
- Medical Record (attributes: record ID, diagnosis, treatment)

Relationship

- A patient can have multiple appointments.
- A doctor can handle multiple appointments.
- Each medical record is associated with one appointment and one doctor, patient.

ERD – Social Media Platform

Entities

- User (attributes: user ID, username, email, join date)
- Post (attributes: post ID, content, post date)
- Comment (attributes: comment ID, content, comment date)
- Like Record (attributes: like ID, like date)

Relationship

- A user can create multiple posts.
- A post can have multiple comments and likes.
- A comment is associated with one user and one post.
- A like is associated with one user and one post.

Week 4

Structured Query Language

SQL – Structured Query Language

SQL is a standard language for accessing and manipulating databases.

What is SQL?

- SQL stands for Structured Query Language
- SQL lets you access and manipulate databases
- SQL became a standard of the American National Standards Institute (ANSI) in 1986, and of the International Organization for Standardization (ISO) in 1987

What SQL can do?

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

SQL Syntax

- **Database Operation** – Create, Use, Drop Database
- **Table Operation** – Create, Drop, Alter(Add Column), Alter(Modify Column), Delete Records Table
- **Data Manipulation** – Insert, Update, Delete Data
- **Querying Data** – Select All Columns, Select Specific Columns, Filtering with WHERE, Sorting with ORDER BY
- **Joins** – Inner Join, Left Join, Right Join, Full Join
- **Constraints** – Primary Key, Foreign Key, Not Null, Unique, Check, Default
- **Views** – Create, Use View

Database Operation

- Create a Database

```
CREATE DATABASE database_name;
```

- Use a Database

```
USE database_name;
```

- Drop a Database

```
DROP DATABASE database_name;
```

Table Operation

- Create a Table

```
CREATE TABLE table_name (  
    column1 datatype CONSTRAINTS,  
    column2 datatype CONSTRAINTS,  
    ...  
    PRIMARY KEY (column_name)  
);
```

Example

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    age INT CHECK (age > 0),  
    email VARCHAR(100) UNIQUE  
);
```

- Drop a Table

```
DROP TABLE table_name;
```

Table Operation

- Alter a Table (Add Column)

```
ALTER TABLE table_name ADD column_name datatype;
```

- Alter a Table (Modify Column)

```
ALTER TABLE table_name MODIFY column_name new_datatype;
```

- Delete ALL Records (data) From a Table (But Keep Structure)

```
TRUNCATE TABLE table_name;
```

- Viewing Table Structure

```
DESCRIBE table_name;
```

- Dropping a Table

```
DROP TABLE table_name;
```

- Dropping a Column

```
ALTER TABLE table_name DROP COLUMN column_name;
```

Data Manipulation

- Insert Data

```
INSERT INTO table_name (column1, column2,...)  
VALUE (value1, value2, ...);
```

Example

```
INSERT INTO Students (student_id, name, age, email)  
VALUE (1, "Thu Huong", 22, "thuhuong@vju.ac.vn");
```

- Update Data

```
UPDATE table_name  
SET column1 = value1, column2 = value2  
WHERE condition;
```

Example

```
UPDATE Students  
SET age = 23  
WHERE student_id = 1;
```

Data Manipulation

- Delete Data

```
DELETE FROM table_name WHERE condition;
```

Example

```
DELETE FROM Students WHERE student_id = 1;
```

Querying Data

- Select All Columns

```
SELECT * FROM table_name;
```

- Select Specific Columns

```
SELECT column1, column2 FROM table_name;
```

- Filtering with WHERE

```
SELECT * FROM table_name WHERE condition;
```

Example

```
SELECT * FROM Students WHERE age > 20;
```

- Sorting with ORDER BY

```
SELECT * FROM table_name ORDER BY column_name ASC|DESC;
```

Example

```
SELECT * FROM Students ORDER BY age DESC;
```

Querying Data

- Grouping with GROUP BY

```
SELECT column_name, COUNT(*) FROM table_name GROUP BY column_name;
```

Example

```
SELECT major, COUNT(*) FROM Students GROUP BY major;
```

Querying Data

- **SELECT syntax**

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition  
GROUP BY column  
HAVING condition  
ORDER BY column [ASC | DESC], column2 [ASC | DESC];
```

```
SELECT column1, column2, ...
```

➔ Specifies the columns to retrieve or expressions to compute

➔ Option

- Use * to select all columns: **SELECT * FROM** table_name;
- Use aliases: **SELECT** column1, column2 **AS** alias_name
- Use functions: **SELECT COUNT**(column1), **AVG**(column2)

```
FROM table_name
```

-> Specifies the table(s) from which to retrieve data

Querying Data

WHERE condition

- ➔ Filters rows based on a condition.
- ➔ Operators: =, <>, <, >, <=, >=, LIKE, IN, BETWEEN, AND, OR, NOT

Example:

```
SELECT name, salary  
FROM employees  
WHERE salary > 5000 AND department = 'IT';
```

GROUP BY column

- ➔ The **GROUP BY** clause is used to group rows that have the same values in specified columns into summary rows (often used with aggregate functions like **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**, etc.)

Querying Data

You have a table called '**Orders**' and you want to find out how many books each customer has ordered.

OrderID	CustomerID	BookID	Quantity
1	101	1	2
2	102	1	3
3	101	2	1
4	103	1	5

```
SELECT
    CustomerID,
    SUM(Quantity) AS TotalBooksOrdered
FROM
    Orders
GROUP BY
    CustomerID;
```

Querying Data

Result

CustomerID	TotalBooksOrdered
101	3 (2+1)
102	3
103	5

- **GROUP BY CustomerID** groups all rows with the same **CustomerID**.
- **SUM(Quantity)** calculates the total quantity of books ordered by each customer.

Querying Data

HAVING condition

➔ The **HAVING** clause is used to filter groups of rows created by **GROUP BY**. It is similar to **WHERE**, but **WHERE** filters rows before grouping, while **HAVING** filters groups after grouping.

```
SELECT
    CustomerID,
    SUM(Quantity) AS TotalBooksOrdered
FROM
    Orders
GROUP BY
    CustomerID
HAVING
    SUM(Quantity) > 4;
```

CustomerID	TotalBooksOrdered
103	5

Querying Data

ORDER BY column1 [ASC | DESC], column2 [ASC | DESC];

➔ The **ORDER BY** clause is used to sort the result set of a query in ascending (**ASC**) or descending (**DESC**) order based on one or more columns. By default, sorting is done in ascending order.

Table: Books

BookID	Title	Author	Price
1	The Great Gatsby	F. Scott Fitzgerald	12.99
2	To Kill a Mockingbird	Harper Lee	10.99
3	1984	George Orwell	9.99
4	Pride and Prejudice	Jane Austen	8.99
5	The Catcher in the Rye	J.D. Salinger	11.99

Querying Data

```
SELECT BookID, Title, Price
FROM Books
ORDER BY Price;
```

BookID	Title	Price
4	Pride and Prejudice	8.99
3	1984	9.99
2	To Kill a Mockingbird	10.99
5	The Catcher in the Rye	11.99
1	The Great Gatsby	12.99

Querying Data

```
SELECT BookID, Title, Author, Price
FROM Books
ORDER BY Author ASC, Price DESC;
```

BookID	Title	Author	Price
3	1984	George Orwell	9.99
1	The Great Gatsby	F. Scott Fitzgerald	12.99
2	To Kill a Mockingbird	Harper Lee	10.99
5	The Catcher in the Rye	J.D. Salinger	11.99
4	Pride and Prejudice	Jane Austen	8.99

- The result are first sorted by **Author** in ascending order (A-Z).
- For books with the same author, they are sorted by **Price** in descending order.

SQL INNER JOIN

```
SELECT columns  
FROM table1  
INNER JOIN table2 ON table1.column = table2.column;
```

- The INNER JOIN command returns rows that have matching values in both tables

Example

```
SELECT employees.name, departments.department_name  
FROM employees  
INNER JOIN departments ON employees.dept_id = departments.id;
```

SQL INNER JOIN

employees

id	name	dept_id
1	Alice	10
2	Bob	20
3	Charlie	10
4	Diana	30
5	Evan	NULL

departments

id	department_name
10	Engineering
20	Marketing
30	Human Resources
40	Sales

Result

name	department_name
Alice	Engineering
Bob	Marketing
Charlie	Engineering
Diana	Human Resources

SQL LEFT JOIN

```
SELECT columns  
FROM table1  
LEFT JOIN table2 ON table1.column = table2.column;
```

- Return all rows from the left table and matched rows from the right table. If no match, NULL is returned for right table columns

Example

```
SELECT employees.name, departments.department_name  
FROM employees  
LEFT JOIN departments ON employees.dept_id = departments.id;
```

SQL LEFT JOIN

employees

id	name	dept_id
1	Alice	10
2	Bob	20
3	Charlie	10
4	Diana	30
5	Evan	NULL

departments

id	department_name
10	Engineering
20	Marketing
30	Human Resources
40	Sales

Result

name	department_name
Alice	Engineering
Bob	Marketing
Charlie	Engineering
Diana	Human Resources
Evan	NULL

SQL RIGHT JOIN

```
SELECT columns  
FROM table1  
RIGHT JOIN table2 ON table1.column = table2.column;
```

- Return all rows from the right table and matched rows from the left table. If no match, NULL is returned for left table columns

Example

```
SELECT employees.name, departments.department_name  
FROM employees  
RIGHT JOIN departments ON employees.dept_id = departments.id;
```

SQL RIGHT JOIN

employees

id	name	dept_id
1	Alice	10
2	Bob	20
3	Charlie	10
4	Diana	30
5	Evan	NULL

departments

id	department_name
10	Engineering
20	Marketing
30	Human Resources
40	Sales

Result

name	department_name
Alice	Engineering
Charlie	Engineering
Bob	Marketing
Diana	Human Resources
NULL	Sales

SQL FULL JOIN

```
SELECT columns  
FROM table1  
FULL JOIN table2 ON table1.column = table2.column;
```

- Return all rows when there is a match in either left or right table. If no match, NULL is filled.

Example

```
SELECT employees.name, departments.department_name  
FROM employees  
FULL JOIN departments ON employees.dept_id = departments.id;
```

SQL FULL JOIN

employees

id	name	dept_id
1	Alice	10
2	Bob	20
3	Charlie	10
4	Diana	30
5	Evan	NULL

departments

id	department_name
10	Engineering
20	Marketing
30	Human Resources
40	Sales

name	department_name
Alice	Engineering
Charlie	Engineering
Bob	Marketing
Diana	Human Resources
Evan	NULL
NULL	Sales

Result

SQL Constraints

- **PRIMARY KEY:** uniquely identifies each record in a table

```
CREATE TABLE table_name (  
    column1 datatype PRIMARY KEY,  
    column2 datatype,  
    ...  
);
```

- **FOREIGN KEY:** ensures referential integrity between two tables

```
CREATE TABLE table1 (  
    column1 datatype PRIMARY KEY,  
    column2 datatype,  
    FOREIGN KEY (column1) REFERENCES table2(column)  
);
```

SQL Constraints

- **NOT NULL:** ensures a column cannot have NULL values

```
CREATE TABLE table_name (  
    column1 datatype NOT NULL,  
    column2 datatype,  
    ...  
);
```

- **UNIQUE:** ensures all values in a column are different

```
CREATE TABLE table_name (  
    column1 datatype UNIQUE,  
    column2 datatype  
);
```

SQL Constraints

- **CHECK:** ensures column values meet a specific condition

```
CREATE TABLE table_name (  
    column1 datatype CHECK(condition),  
    column2 datatype,  
    ...  
);
```

- **DEFAULT:** set a default value if no value is specific

```
CREATE TABLE table_name (  
    column1 datatype DEFAULT default_value,  
    column2 datatype  
);
```

SQL Views – CREATE VIEW

- Create a virtual table from a query

```
CREATE VIEW view_name AS  
SELECT columns  
FROM table  
WHERE condition;
```

Example

```
CREATE VIEW active_customers AS  
SELECT name, email  
FROM customers  
WHERE status = 'active';
```

SQL CREATE VIEW

customers

id	name	email	status
1	John Smith	john@email.com	active
2	Jane Doe	jane@email.com	inactive
3	Bob Johnson	bob@email.com	active
4	Alice Brown	alice@email.com	active

Result

name	email
John Smith	john@email.com
Bob Johnson	bob@email.com
Alice Brown	alice@email.com

```
CREATE VIEW active_customers AS  
SELECT name, email  
FROM customers  
WHERE status = 'active';
```

SQL Views – USE VIEW

- Query a view like a regular table

```
SELECT * FROM view_name;
```

Example

```
SELECT * FROM active_customers;
```

name		email
John Smith		john@email.com
Bob Johnson		bob@email.com
Alice Brown		alice@email.com

SQL Views – UPDATE VIEW

- Modifies an existing view

```
CREATE OR REPLACE VIEW view_name AS  
SELECT new_columns  
FROM table  
WHERE new_condition;
```

Example

```
CREATE OR REPLACE VIEW active_customers AS  
SELECT name, email, phone  
FROM customers  
WHERE status = 'active' AND last_purchase_date > '2023-01-01';
```

SQL Views – UPDATE VIEW

customers

id	name	email	phone	status	last_purchase_date
1	John Smith	john@email.com	555-0101	active	2023-06-15
2	Jane Doe	jane@email.com	555-0102	inactive	2022-12-01
3	Bob Johnson	bob@email.com	555-0103	active	2023-03-20
4	Alice Brown	alice@email.com	555-0104	active	2024-01-10

Result

active_customers view:

name	email	phone
John Smith	john@email.com	555-0101
Bob Johnson	bob@email.com	555-0103
Alice Brown	alice@email.com	555-0104

```
CREATE OR REPLACE VIEW active_customers AS
SELECT name, email, phone
FROM customers
WHERE status = 'active' AND last_purchase_date > '2023-01-01';
```

SQL Views – DROP VIEW

- Deletes a view

```
DROP VIEW view_name;
```

Example

```
DROP VIEW active_customers;
```

SUMMARY TABLE

Concept	Key Syntax	Purpose
INNER JOIN	INNER JOIN ON	Returns matching rows
LEFT JOIN	LEFT JOIN ON	All left rows + matched right rows
RIGHT JOIN	RIGHT JOIN ON	All right rows + matched left rows
FULL JOIN	FULL JOIN ON	All rows from both tables
PRIMARY KEY	PRIMARY KEY	Uniquely identifies a record
FOREIGN KEY	FOREIGN KEY REFERENCES	Links two tables
NOT NULL	NOT NULL	Prevents NULL values
UNIQUE	UNIQUE	Ensures all values are distinct
CHECK	CHECK (condition)	Validates data condition
DEFAULT	DEFAULT value	Sets a default value
CREATE VIEW	CREATE VIEW AS	Creates a virtual table
USE VIEW	SELECT FROM view	Queries a view
DROP VIEW	DROP VIEW	Deletes a view

Week 5

Database Management

WHAT IS DATABASE MANAGEMENT

- Database Management allows a person to organize, store, and retrieve data from a computer.
- Database Management can also describe the data storage, operations, and security practices of a database administrator (DBA) throughout the life cycle of the data.
- Managing a database involves designing, implementing, and supporting stored data to maximize its value. Database Management Systems, according to the DAMA DMBOK, include various types:
 - **Centralized:** all the data lives in one system in one place. All users come to that one system to access the data.
 - **Distributed:** Data resides over a variety of nodes, making quick access possible.
 - **Federated:** Provisions data without additional persistence or duplication of source data. It maps multiple autonomous databases into one large object. This kind of database architecture is best for heterogeneous and distributed integration projects. Federated databases can be categorized as:
 - Loosely Coupled
 - Tightly Coupled

WHAT IS DATABASE MANAGEMENT

- Database Management allows a person to organize, store, and retrieve data from a computer.
- Database Management can also describe the data storage, operations, and security practices of a database administrator (DBA) throughout the life cycle of the data.
- Managing a database involves designing, implementing, and supporting stored data to maximize its value. Database Management Systems, according to the DAMA DMBOK, include various types:
 - **Centralized:** all the data lives in one system in one place. All users come to that one system to access the data.
 - **Distributed:** Data resides over a variety of nodes, making quick access possible.
 - **Federated:** Provisions data without additional persistence or duplication of source data. It maps multiple autonomous databases into one large object. This kind of database architecture is best for heterogeneous and distributed integration projects. Federated databases can be categorized as:
 - Loosely Coupled
 - Tightly Coupled

DATABASE ADMINISTRATOR (DBA)

- A database administrator, or DBA, is responsible for *maintaining, securing, and operating* databases and also ensures that data is correctly stored and retrieved.
- DBAs often work with developers to design and implement new features and troubleshoot any issues. A DBA must have a strong understanding of both technical and business needs.
- The role of DBA is becoming increasingly important in today's information-driven business environment. Throughout the world, more and more organizations depend on data to discover analytical insights on market conditions, new business models, and cost-cutting measures. The global cloud computing market is also expected to expand as companies move their business operations to the cloud. ***Consequently, the need for qualified DBAs will only continue to grow.***

RESPONSIBILITY OF (DBAs)

- The specific responsibilities of a database administrator vary depending on the size and needs of the organization they work for. However, most DBA duties will include developing and maintaining databases, ensuring data security, tuning performance, backing up data, and providing training and support to users. DBAs may also be responsible for designing databases and overseeing their construction in larger organizations.
 - Data Modeling: Defining schemas and relationships.
 - Backup and Recovery: Implementing regular backups and disaster recovery plans.
 - Performance Tuning: Indexing, query optimization, and load balancing to enhance efficiency.

TYPEs OF DATABASE ADMINISTRATORs (DBAs)

There are several types of database administrators, each with specific duties and responsibilities. The most common types of DBAs include **system administrators**, **database architects**, **database analysts**, **data modelers**, **application DBAs**, **task-oriented DBAs**, **performance analysts**, **data warehouse administrators**, and **cloud DBAs**.

- **System administrators** are responsible for the overall management and upkeep of a computer system, including installing and configuring software, applying security patches, and monitoring system performance.
- **Database architects** design databases to meet the specific needs of an organization.
- **Database analysts** collect and analyze data to help improve database performance. They may also be responsible for developing reports and providing recommendations to database administrators.
- **Data modelers** create and maintain data models that depict the relationship between data elements. Data modeling is a critical component of effective database design.
- **Application DBAs** are responsible for administering databases that support applications. Specific tasks include installing and configuring applications, ensuring that data is synchronized correctly between databases, and troubleshooting application-related issues.

TYPES OF DATABASE ADMINISTRATORS (DBAs)

- **Task-oriented DBAs** focus on a particular area of database administration, such as backup and recovery, security, or performance tuning. They typically have in-depth knowledge of a specific database management system (DBMS).
- **Performance analysts** monitor database performance and identify areas where improvement is needed. They may also be responsible for creating performance reports and providing recommendations to database administrators.
- **Data warehouse administrators** manage databases that store data for business intelligence or decision-support applications. They are responsible for extracting data correctly, transforming the data, and loading it into the data warehouse.
- **Cloud DBAs** are responsible for administering databases hosted in a cloud computing environment, provisioning and managing database instances, setting up replication and high availability, and monitoring database performance.

TYPES OF DATABASE ADMINISTRATORS (DBAs)

- **Task-oriented DBAs** focus on a particular area of database administration, such as backup and recovery, security, or performance tuning. They typically have in-depth knowledge of a specific database management system (DBMS).
- **Performance analysts** monitor database performance and identify areas where improvement is needed. They may also be responsible for creating performance reports and providing recommendations to database administrators.
- **Data warehouse administrators** manage databases that store data for business intelligence or decision-support applications. They are responsible for extracting data correctly, transforming the data, and loading it into the data warehouse.
- **Cloud DBAs** are responsible for administering databases hosted in a cloud computing environment, provisioning and managing database instances, setting up replication and high availability, and monitoring database performance.

Role of a DBA Evolved With Cloud Computing?

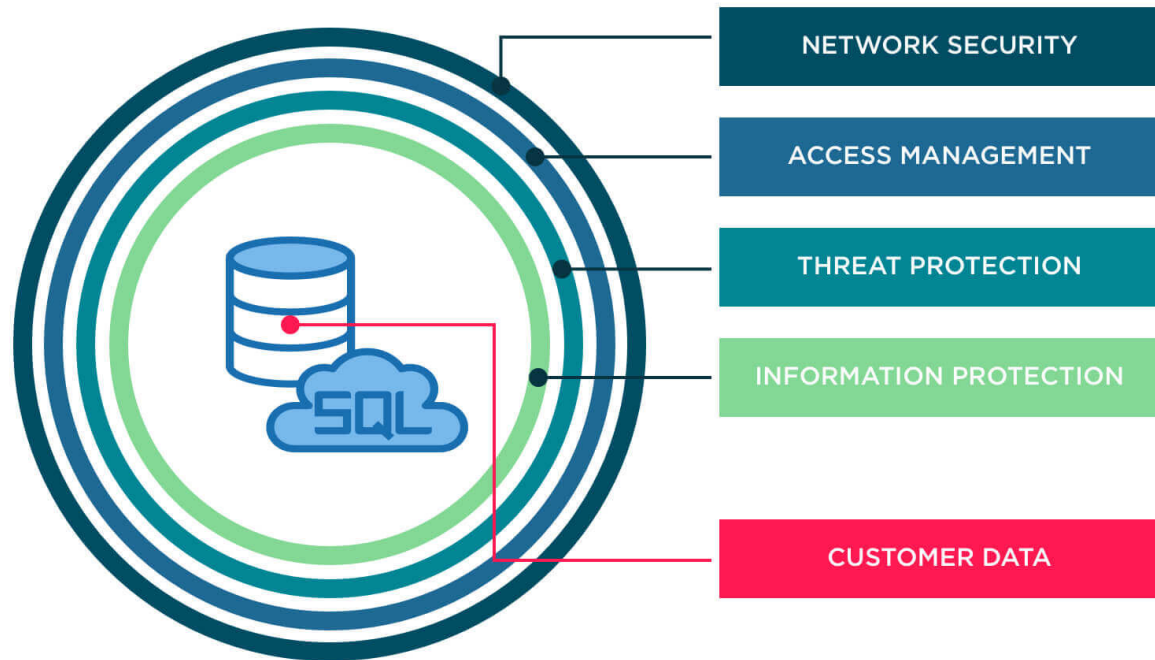
- The role of a database administrator has evolved significantly with the advent of cloud computing. Rather than being responsible for managing on-premises hardware and software, DBAs now need to be able to work with cloud-based platforms. This requires a different set of skills and knowledge and a different approach to work.
- DBAs need to be able to work with different types of databases, such as MySQL, MongoDB, and Cassandra. They also need to be familiar with cloud-based tools and platforms, such as Amazon Web Services (AWS) and Microsoft Azure.
- One of the most significant changes is that DBAs are no longer responsible for managing the underlying infrastructure. With cloud computing, this is all managed by the provider. As a result, DBAs now perform more strategic tasks, such as data analytics, user experience design, and cybersecurity. DBAs often work directly with users and business leaders on developing new ways to use data and software to automate processes, reduce costs, and stay competitive.
- This requires a new set of skills from DBAs. In the past, having strong technical skills was the most important requirement. There is less need for these skills with cloud computing. Instead, DBAs need to communicate and collaborate with users to understand their needs and business environment. They also need to work with other teams, such as DevOps, to help deliver software that will solve business problems.

Role of a DBA Evolved With Cloud Computing?

Overall, the traditional role of a DBA is changing significantly thanks to cloud computing. DBAs need to be able to adapt to these changes to be successful in their roles.

SECURITY MANAGEMENT

- Database security is a set of practices and technologies used to protect database management systems from malicious cyberattacks and unauthorized use. Database security is a complex task that combines several information security disciplines—application security, data security, and endpoint security.

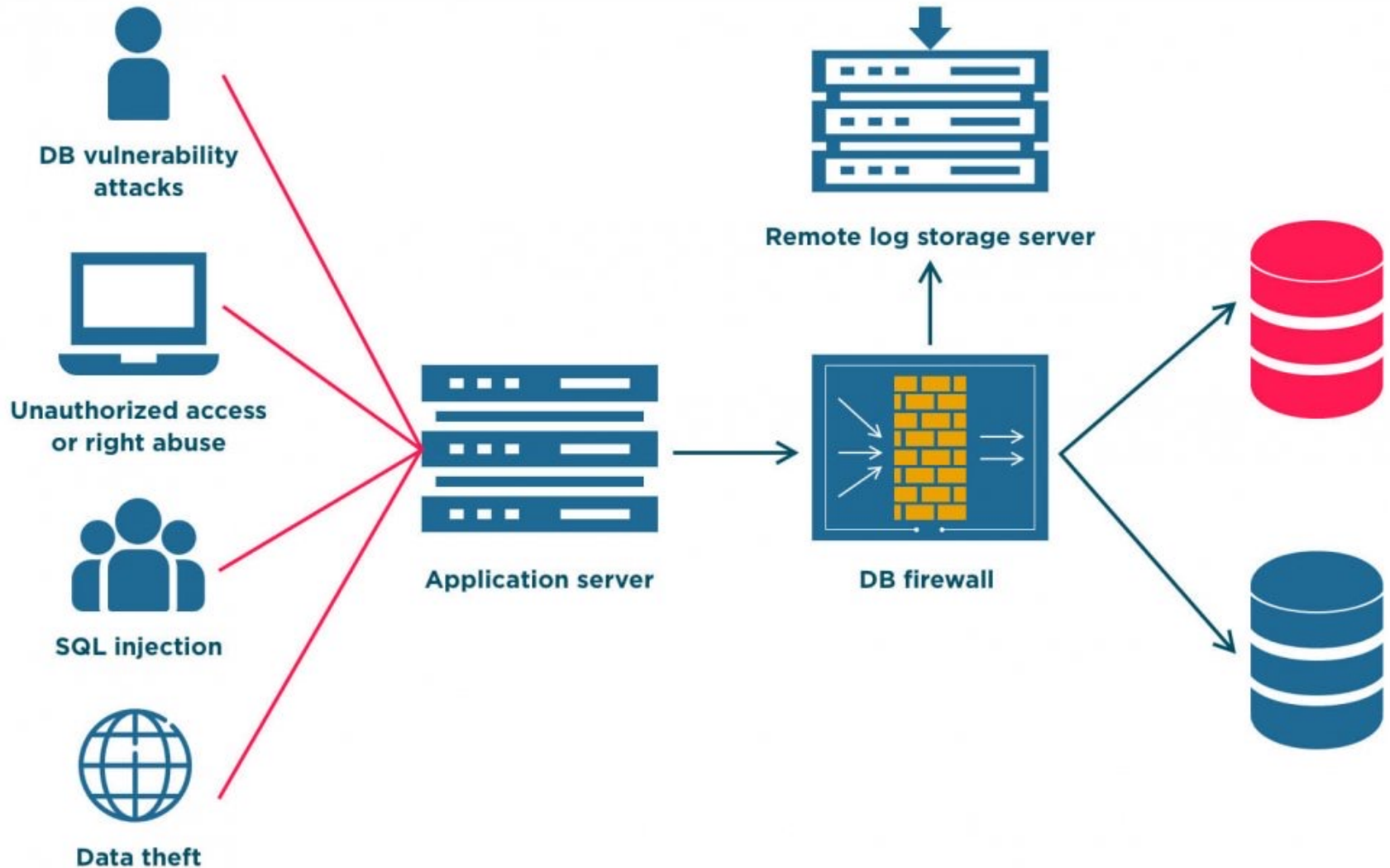


- The goal of database security is to protect against misuse, data corruption, and intrusion, not only of the data in the database, but of the data management system itself and applications that access the database. Another aspect of database security is protecting and hardening the physical or virtual server hosting the database, and the surrounding computing and network environment.

Main Components of database security

- Setting up security controls: database management systems usually offer custom systems of security assurance and monitoring. Developers need to edit the settings.
- Configuration of a DBMS: security requires an organized data structure, edited safety settings, permissions, and access controls.
- Authentication: each database should be automatically verifying user access to settings, networks, and data.
- Constant database audit: monitoring database safety helps to catch security errors in time, reducing the impact of a breach, and detect suspicious patterns.
- Backups: a safe database is always an audited mechanism for backup creation and management. Even if data is compromised or deleted, you will always have a copied version.
- Encryption: implementing file encryption protects the contents of the file from being understood and utilized by cybercriminals. Even if they get access to the database, the information itself won't be displayed.

Database Security



Database Security Threats

- Insider threats: It's the simplest type of a database threat but also the one which is the hardest to predict. Basically, a person who has assigned access to the database and its settings leaks, damages, or deletes information – and there's a number of ways how it could be done.

4 indicators of a malicious insider



Disgruntlement



**Unusual
enthusiasm**



**Unusually frequent
trips and vacations**



**Unexplained changes in
financial circumstances**

Database Security Threats

Human error: similar to insiders' negligent threats. Usually, they happen when an employee's social media account, email, device, or used software is compromised. The virus hijacks the system and receives access to the database as well.

How to prevent human errors



Update
security policy



Educate
employees



Enforce the principle
of least privilege



Monitor
employee activity

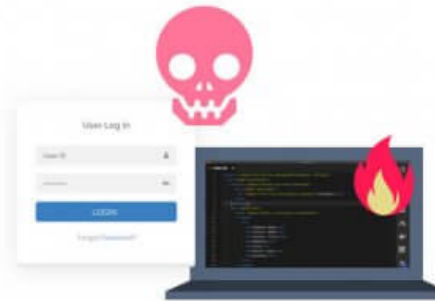
Database Security Threats

Injection attacks: If, in the previous two, any device, file, or software can be targeted, injection attacks are carried out specifically in a database or database management system. A cybercriminal inserts attack strings written in SQL or NoSQL into a database. The HTTP protocols and data layers of the application are compromised – since fake strings are seen by the system as the real ones.

WHAT IS SQL INJECTION?



A SQL query is one way an application talks to the database



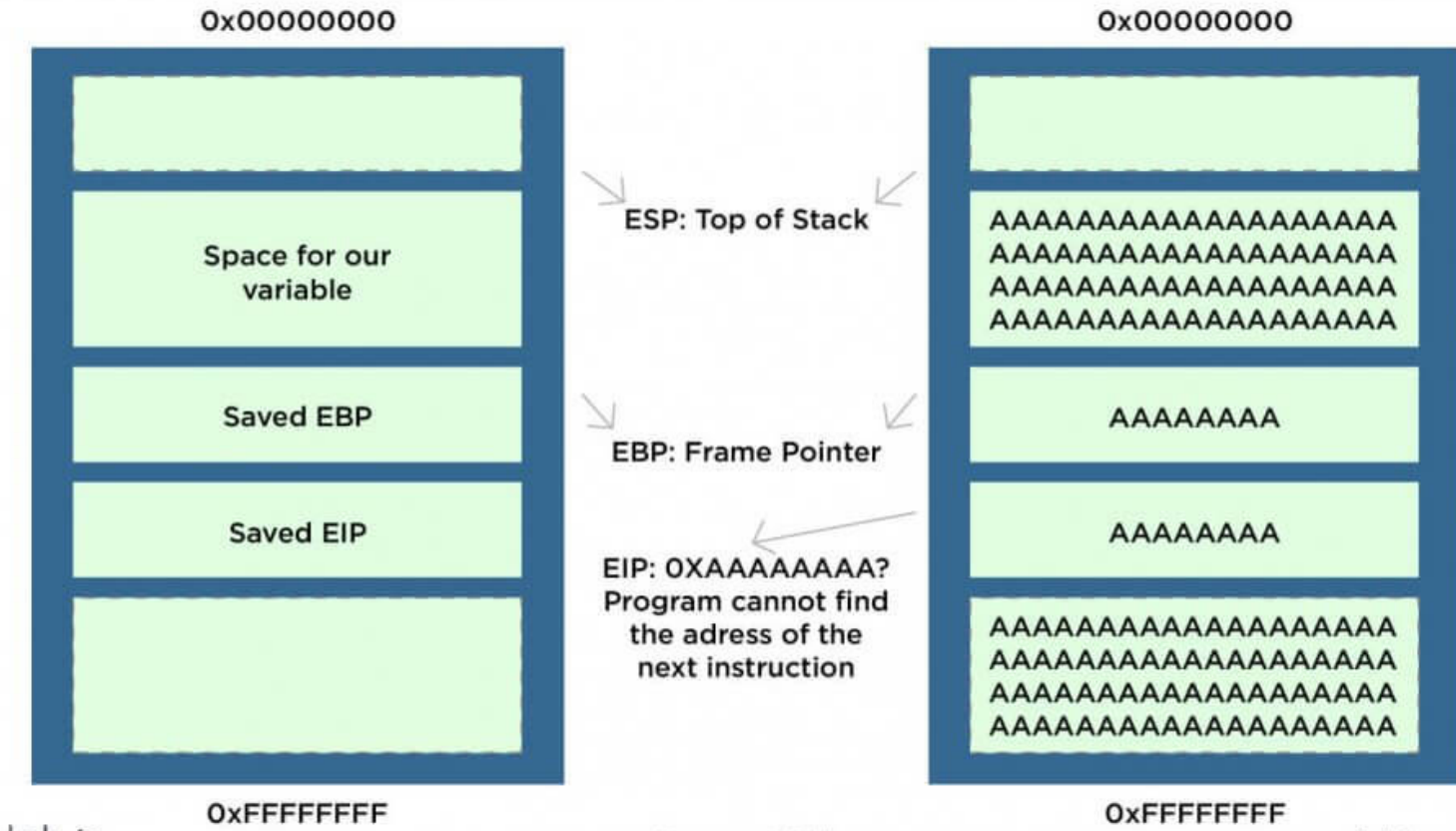
SQL injection occurs when an application fails to sanitize untrusted data (such as data in web form fields) in a database query.



An attacker can use specially-crafted SQL commands to trick the application into asking the database to execute unexpected commands.

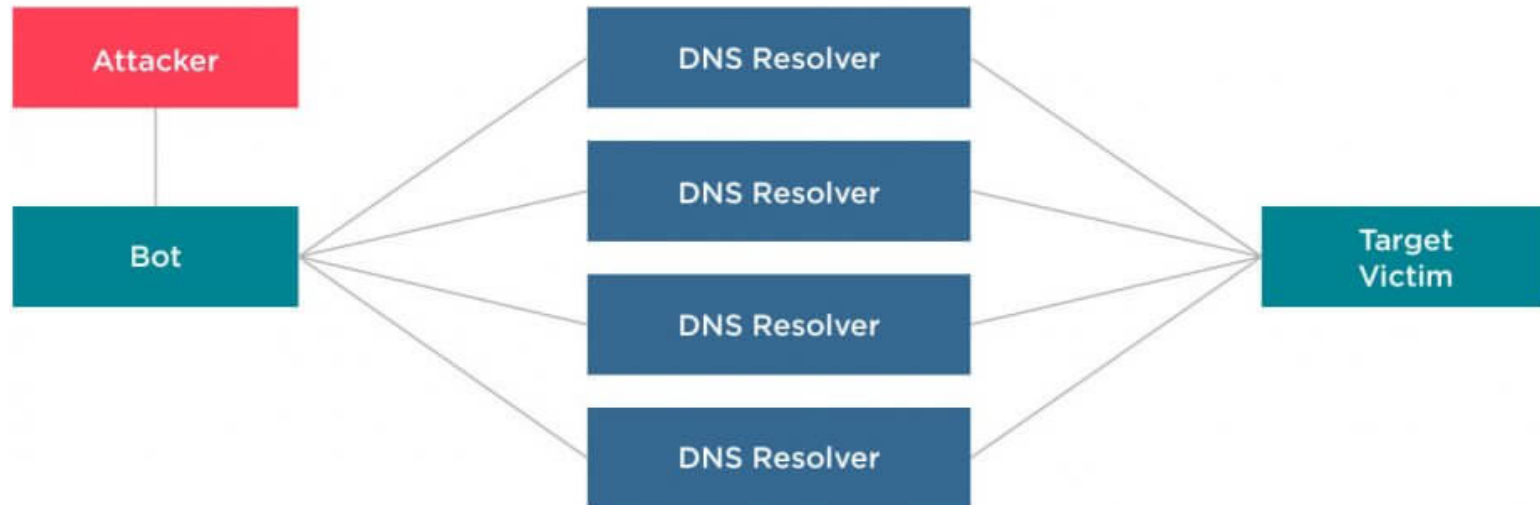
Database Security Threats

Overflow exploitation: Sometimes the goal of security attack isn't leaking the information but blocking the performance of the database – which will lead to compromised app performance. For instance, it can be performed on the e-commerce platform during a busy season in order to block check-ups, payment transfers, and registration.



Database Security Threats

Denial of service (DOS) attacks: If an attacker has a goal of blocking database performance, it can be done with many requests, not necessarily the long ones, like in buffer overload. At some point, the server can no longer handle that many data processing requests and shuts down. As a result of downtime, the company loses potential customers, revenue, and risks its reputation.

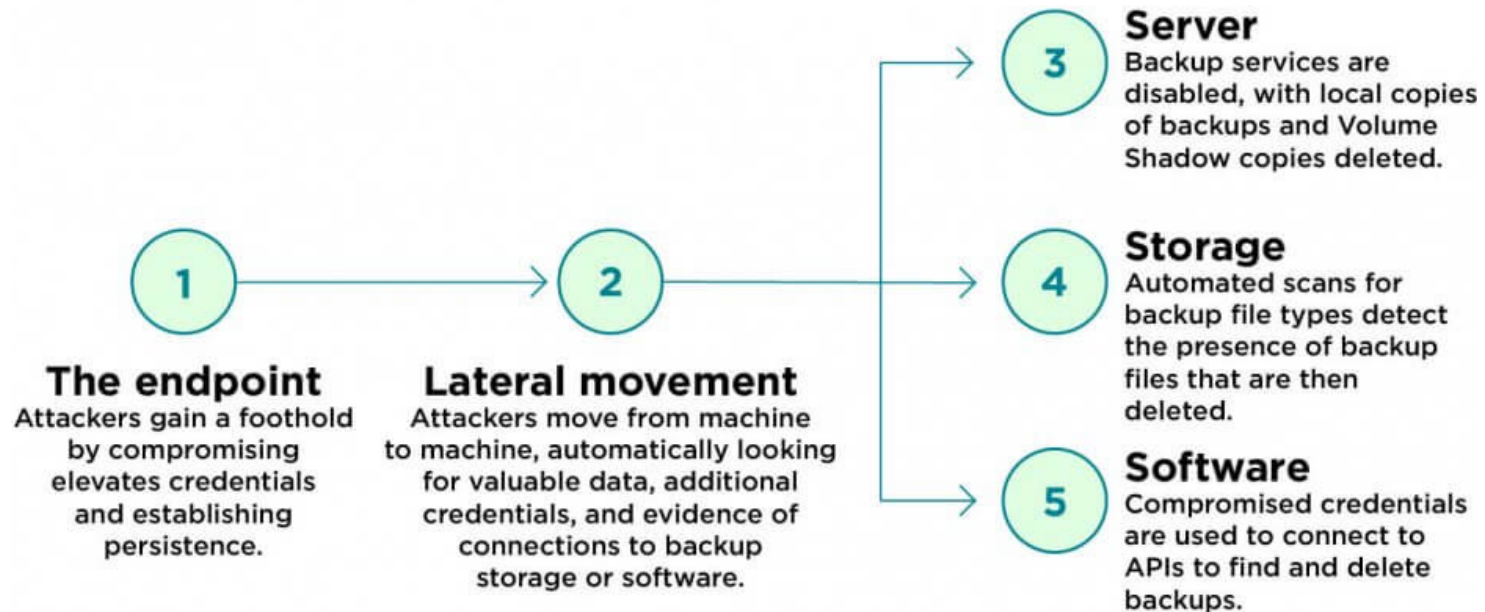


Database Security Threats

Backup attacks: Organizations usually aren't too careful about securing their backup files. Often, they use the same methods and controls as the main database. If the main contents were compromised, an attacker wouldn't take long to crack up the backup. Another common mistake is failing to protect the backup files altogether or not tracking their location properly.

The backup attack kill chain

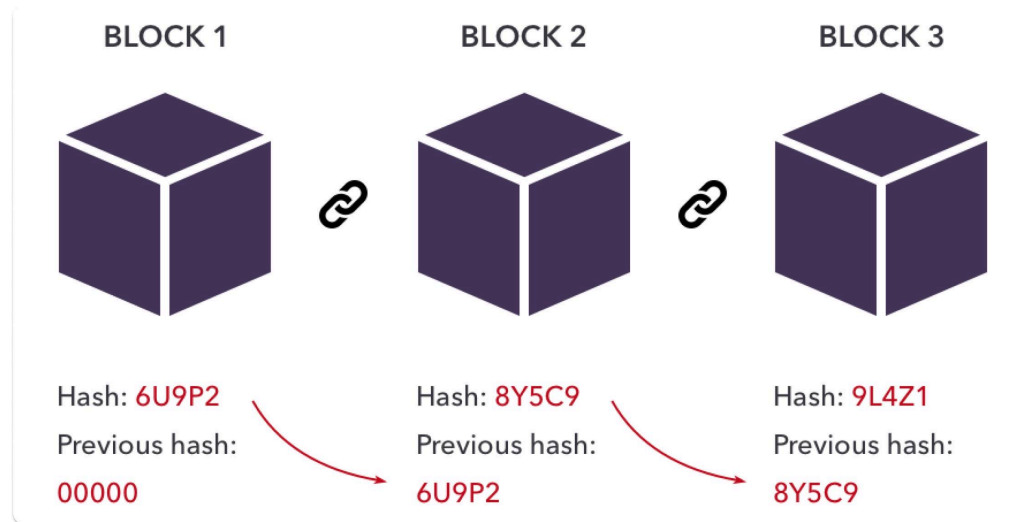
Malware intent on finding and deleting backups takes a very specific path - one that, once understood by IT, can be used to strengthen the security of backups.



Blockchain Database

Definition:

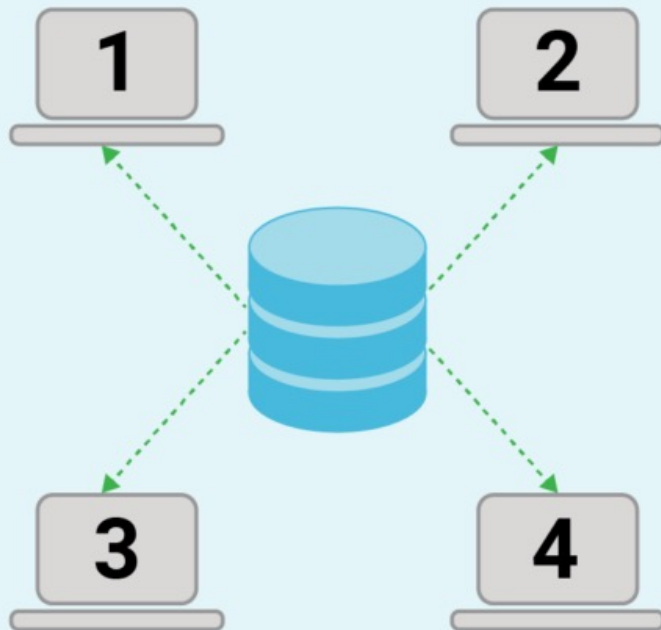
- A decentralized, distributed ledger that records transactions across multiple computers.
- Data is stored in blocks linked together in a chain.
- Each block in the chain contains a record of multiple database transactions, and every block is linked to the one that came before it. This creates a continuous, unbroken chain of information.
- Key features: Immutable, Transparent, Secure, and Decentralized.
- Because blockchain distributed databases are decentralized and encrypted, they are often used for secure and transparent record-keeping in industries like finance, healthcare, and supply chain management. Cryptocurrencies like Bitcoin also use blockchain technology as the foundation of their transaction systems.



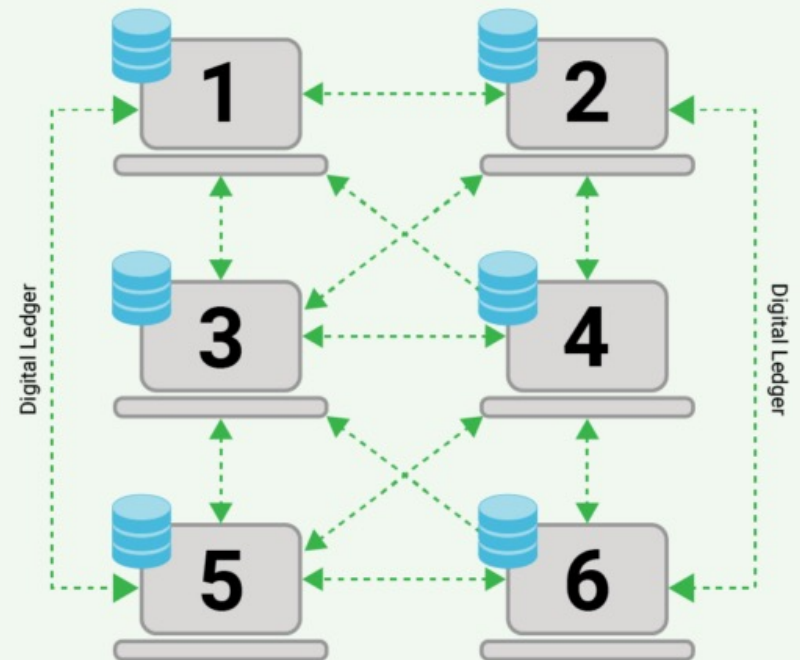
Blockchain Database vs Traditional Database

CENTRALIZED DATABASE vs BLOCKCHAIN

Centralized Database



Blockchain Database



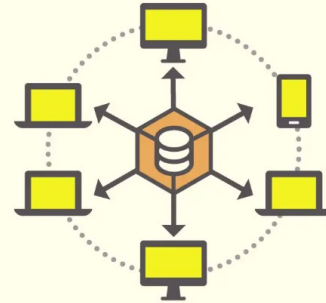
Blockchain Process



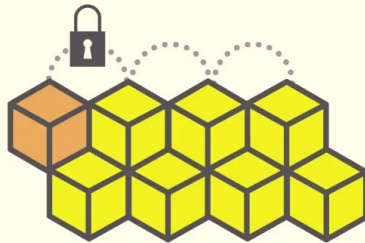
**New data
(e.g. a transaction)
is entered into the
blockchain.**



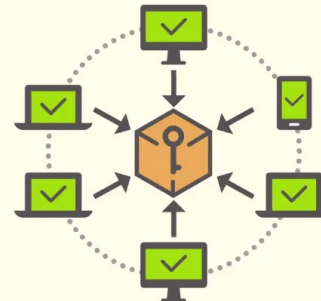
**A block
representing this
data is created.**



**The block is
broadcast to all
the nodes in the
blockchain network.**



**If approved, the new
block is added to the
chain permanently.**



**Each node (participant)
chooses to approve or deny
the new block.**

How Blockchain Works as a Database

Components:

- Block: Store batches of transactions/data.
- Hashes: Unique fingerprints (SHA-256) for each block.
- Consensus mechanisms: Rules for validating transactions (PoW, PoS, etc.).
- Nodes: Computers maintaining the blockchain (Full Nodes, Light Nodes).

Data Structure – Each block contains:

- Previous Block's Hash (links blocks in order).
- Timestamp (when the block was created).
- Transaction Data (or any stored information).
- Nonce (used in mining for Proof-of-Work).

Blockchain Database Use Case

Some common use cases for blockchain databases:

- Blockchain – Software that captures transactions in an electronic ledger using cryptography for privacy and consistency and “distributed” among persons with access (public or private), eliminating intermediaries.
- Blockchain/Crypto – Blockchain software used to capture transactions of digital “crypto” currencies.
- Blockchain/Smart Contract – Computer programs on a blockchain that, when certain conditions or user actions occur, perform a function such as accessing data from a database, triggering next action, or automating a workflow.

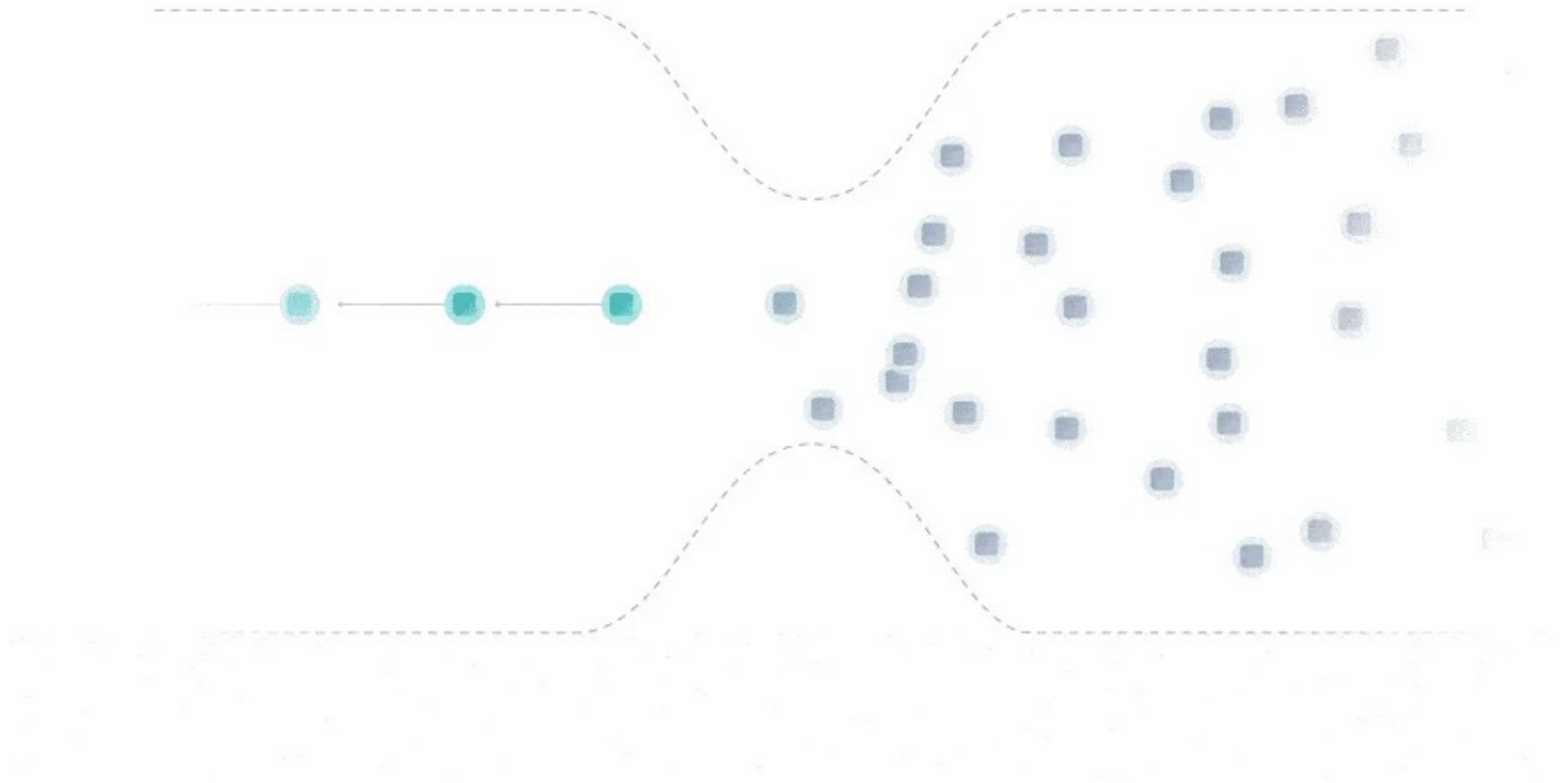
Blockchain Database - Tangle

Tangle is a new type of distributed ledger technology that's different than the traditional blockchain model.

- Blockchains and the Tangle both fall under the same top-level category: DLT. The main difference between the Tangle and a blockchain lies in its data structure, which gives the Tangle the following unique features:
 - No transaction fees.
 - No miners.
- The blockchain data structure consists of a chain of sequential blocks, where each block contains a limited number of transactions. As a result, you can attach new transactions to only one place: A block at the end of the chain.
- To secure the network against attacks, each block in a blockchain must be mined. Mining requires a lot of computational power, which is why miners are incentivized by transaction fees and block rewards.

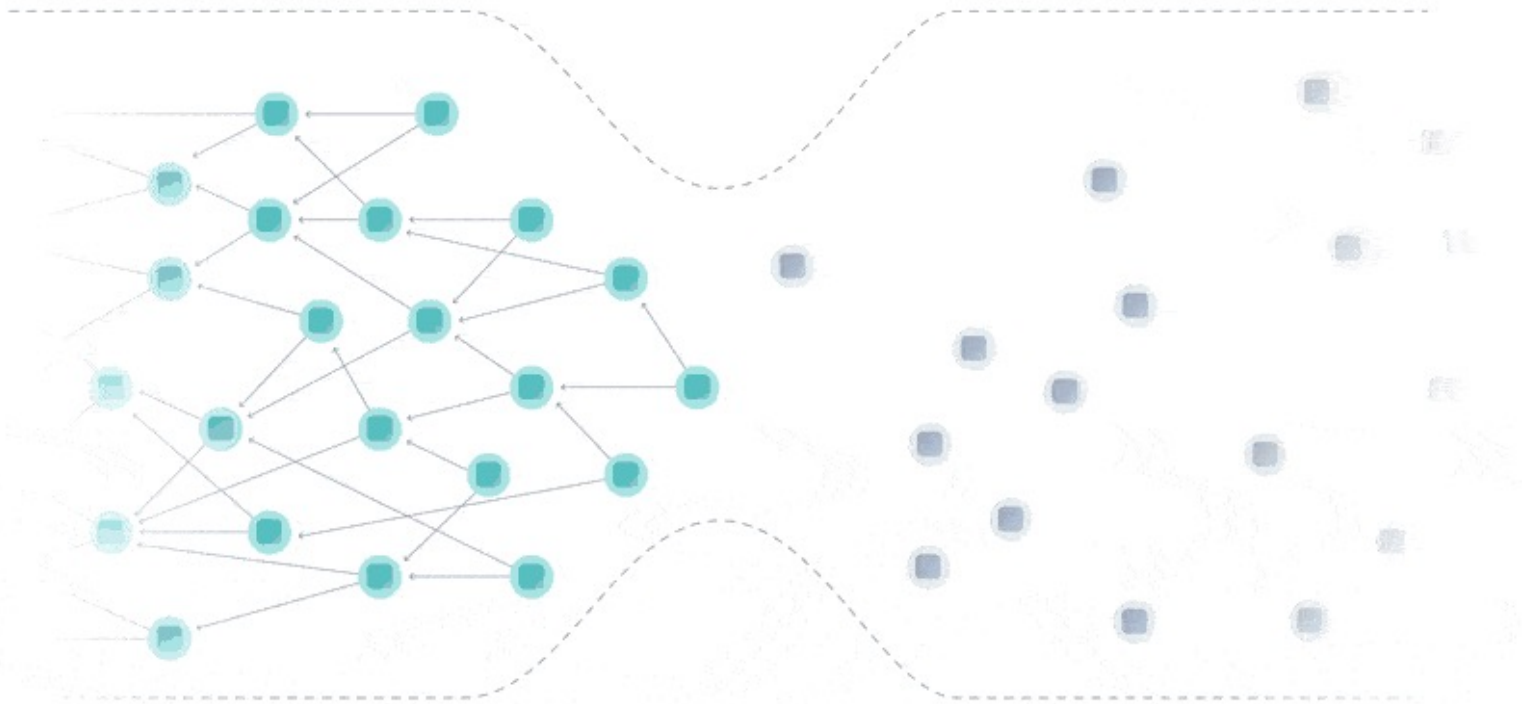
Blockchain Database - Tangle

- Due to this block limitation known as the blockchain bottleneck, blockchain networks often experience slow confirmation times and high transaction fees.



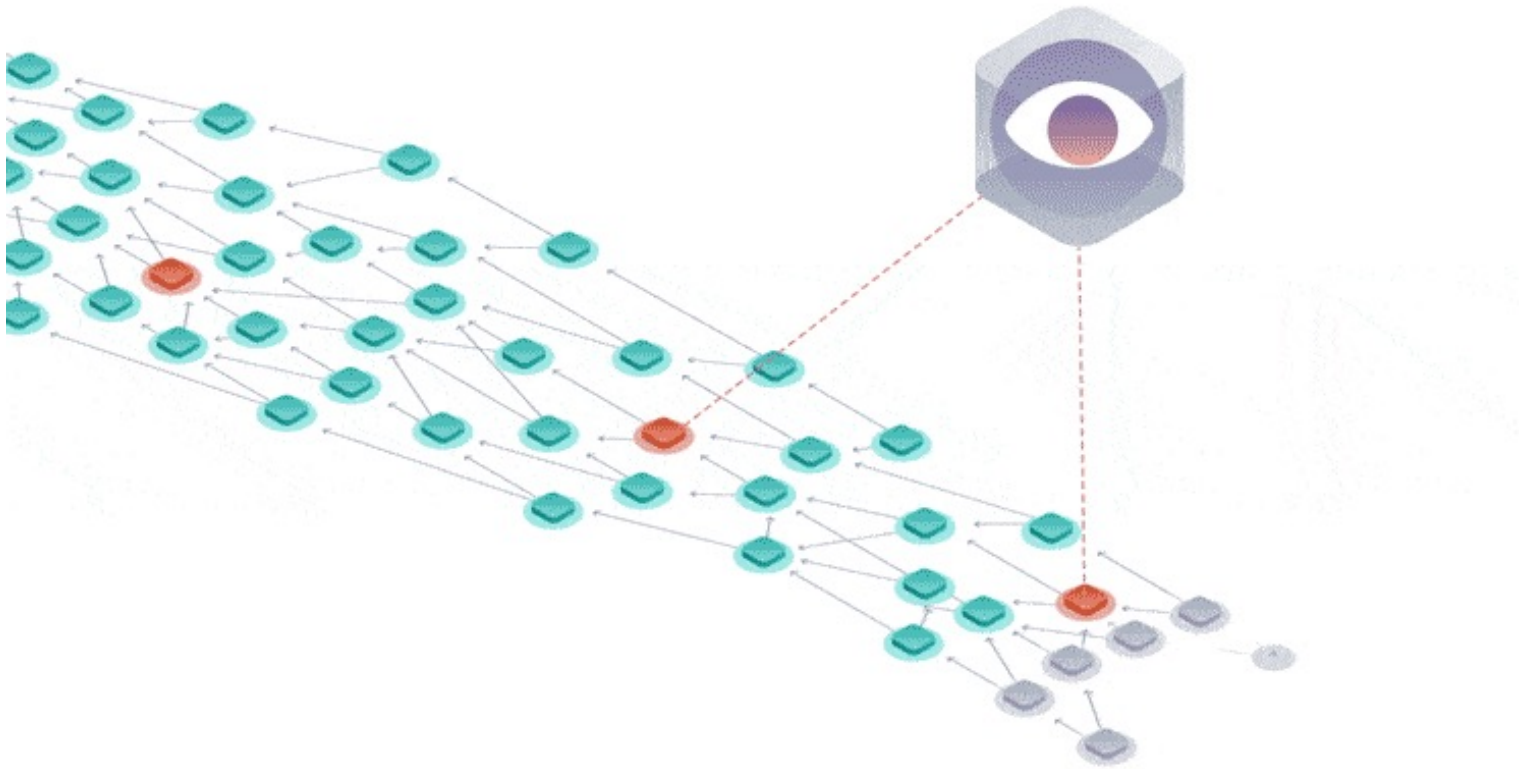
Blockchain Database - Tangle

- The Tangle data structure is a directed acyclic graph (DAG), where each transaction approves two previous ones. Rather than being limited to a single place for attaching new transactions, you can attach transactions anywhere in the Tangle, which drastically reduces the limit on confirmation times.



Blockchain Database - Tangle

- Instead of having miners, the Tangle is secured by a temporary finality device called the Coordinator that decides which parts of the Tangle are valid and should be confirmed.



How to Create Blockchain Database

Using blockchain as a database involves several complex steps:

- **Identify/define the requirements and purpose of the database.** First, determine the specific purpose and requirements of the database. This includes identifying the type(s) of data to be stored, the level of security required, and the users who will have access to the database.
- **Choose a blockchain platform.** There are several blockchain platforms that can serve as the database platform, each with unique features. Some examples include Ethereum, Hyperledger, and Corda.
- **Define the data schema.** Next, define the data schema. Specify the structure and format of the data that will be stored in the database, such as data types, fields, and relationships between data.
- **Develop the smart contract to control the database.** A smart contract is a self-executing program that runs on the blockchain platform and controls interactions between users and the database. Typically written in a programming language such as Solidity or Java, the smart contract defines the rules and logic for accessing and modifying the data in the database.

How to Create Blockchain Database

Using blockchain as a database involves several complex steps:

- **Deploy the database on the blockchain platform.** Once the smart contract has been developed, it can be deployed on the platform. This creates a decentralized database that network users can access and modify.
- **Test and maintain the database.** Test the database thoroughly to ensure it functions correctly and meets all of your requirements. Ongoing maintenance is essential to ensuring the security and integrity of the database over time.